Cluster Computing (2026) 29:790
https://doi.org/10.1007/s10586-026-06545-4

Implementing machine learning model drift detection in fog computing architectures for remote healthcare systems

Salman Ahmed[1] · **Nayyer Masood**[1]

Received: 21 July 2025 / Revised: 18 July 2026 / Accepted: 3 September 2026
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract
Machine learning (ML) is an active area of research for solving classification and regression problems. ML models are used in numerous fields ranging from medicine to engineering, social science, and business. When deployed after training, these models generate predictions to support decision-making. However, trained models may perform less effectively on new, unseen data than during training; their performance may deteriorate over time. Such deterioration can result from concept drift, where the underlying data distribution changes over time and begins to exhibit patterns that differ from those in the training data. This is especially critical in distributed environments such as fog computing, where drift may occur independently across multiple fog devices. In this paper, we focus on drift detection for real-time healthcare environments, where a deployed ML model (typically a classifier) processes unlabeled data. Our Real-time Drift Detection Method (RTDDM) detects drift by integrating supervised and unsupervised learning: we simultaneously train a classifier and fit a clustering model on the same dataset, thereby linking classifier accuracy to cluster purity. During deployment, we compare the classifier's predicted label for each unlabeled instance with the class label assigned to its cluster, yielding an approximate error signal that can be used for drift detection. The experiments show that our proposed method reduces the need for human intervention.

Keywords Machine learning · Model drift detection · Real-time drift detection · Drift detection for unlabeled data · Concept drift in classifiers

1 Introduction

Today, many of the software applications from banking, telecom, e-commerce, healthcare systems, etc. adopt machine learning techniques for better decision making and future planning. These techniques help different systems in different ways. For example, in banking systems, machine learning can be used to predict that if a customer has the capacity to enrol in a loan programme or not, telecom companies to predict churn customers, e-commerce owners to predict the upcoming trend of saleable products by visualising customer behaviours, and healthcare systems to predict critical health timely to prevent severe causes. However, with the passage of time, the decision-making validity of these systems is reduced because of random changes in the input data of the systems or the addition of some new parameter. This is because of concept drift, when the trained model generated by the machine learning (ML)algorithm loses performance by predicting wrong target label for unknown data. The key cause of this inaccuracy arises when real-time data is evolving over time to generate different data patterns compared to the data on which the ML model is trained.

In healthcare, various changes in health data can cause concept drift. Some are discussed below:

1. Demographics: Changes in the patient population, such as an increase in the average age of patients or a shift in the proportion of male to female patients, can cause a concept drift[1, 2].

Salman Ahmed and Nayyer Masood contributed equally to this work.

✉ Salman Ahmed
salman.ahmed@cust.edu.pk

Nayyer Masood
nayyerdar64@gmail.com

[1] Department of Computer Science, Capital University of Science and Technology, Zone-V, Kahuta Road, Islamabad 44000, Pakistan

Published online: 25 September 2026

2. Medical treatments or technology: Changes in medical treatments, such as the introduction of new drugs, surgical procedures, or diagnostic technologies, can cause a drift of concepts [3].
3. Disease progression: Changes in symptoms or complications associated with a certain disease can cause drift of concepts [4, 5].
4. Patient behaviour: Changes in the way patients seek medical attention or changes in their adherence to medication regimens can cause a drift of the concept [6].
5. Environmental factors: Changes in the environment, such as increased air pollution or increased prevalence of certain infections, can cause a concept drift [7].

Under parameter 3 (progression of the disease) in the previous list, if we consider a model trained by any machine learning algorithm for the symptoms of Influenza (flu) based on features such as fever, cough, shortness of breath, fatigue (tiredness), sore throat, runny nose, muscle pain, headache. This model is considered outdated after COVID-19 due to similar features for COVID-19 as well. Influenza (flu) and COVID-19 are both contagious respiratory diseases but are caused by different viruses. However, change or loss of smell and taste is normally associated with COVID-19 [4]. So in this context, the machine learning model needs to be re-trained with new features and labels. Similarly, ECG changes have been observed in heart patients over time that could lead patients to the critical stage [8].

There are several definitions exist in literature for data drift, concept drift and model drift. Data drift refers to simply change in data distribution, concept drift refers to the update in association of target attribute with features, and model drift refers to degradation of performance in ML model. A real-world example from Diabetes 130-US hospitals is analysed for the 1999-2008 dataset [9]. This dataset keeps records of readmission of patients in hospitals. Suppose that we have a subset of 1000 samples, each with four characteristics: age, weight, race, and serum glucose level, and a binary target variable y (hospital readmission in 30 days).

Initially, the collected data can follow the pattern governed by the following equation: y = 1 if age>= 60 and weight>= 200 and race = “African American" and serum glucose level>= 200 y = 0 otherwise. This equation represents the relationship between the features and the target for the first 400 samples in the subdata set, and the model trained on these data would learn to predict hospital readmission within 30 days for patients with age>= 60, weight>= 200, African American race and serum glucose level>= 200. However, the relationship between the features and the target may change over time. For example, the next 600 samples in sub dataset may be generated according to the following equation: y = 1 if age>= 55 and weight>= 220 and race = “African American" and glucose serum level>= 180 y = 0 otherwise. This alteration of quantitative association between the predictor variables and the dependent variable constitutes an instance of concept drift, as the model trained on the first 400 samples would not perform well on the next 600 samples. The model would need to be updated or retrained to reflect these changes in-order to make accurate predictions for the next 600 samples.

There are commonly three categories identified as data distribution-based, error rate-based, and Statistical Ensemble Drift Detection [10]. For reliable model performance, there is a need to integrate drift detection methods into the sensitive systems like Remote Health Monitoring Systems (RHMS) to preserve ML prediction accuracy.

In RHMS, where patients are physically outside the reach of doctors, we need a mature prediction system to monitor patient health. In this type of system, the fog computing-based approach [11] is more suitable than the cloud-based approach as this is closer to the patient. This approach enables real-time analysis and decision-making based on local resources [12]. Fog architecture has many advantages for applications that need real-time response, such as remote health monitoring systems. Low network latency, low bandwidth utilization, and low power consumption are some of the parameters that distinguish the fog computing architecture from the cloud computing architecture.

This research is part of Erasmus+ funded project SAFE-RH (Sensing, ArtiFicial intelligence, and Edge networking toward rural health monitoring). The project aims to detect, diagnose and monitor different health-related issues such as falls, fall detection, arrhythmia, and fetal movement etc for the patients in remote areas. The project connects doctors and patients remotely; however, the main objective is to facilitate those rural BHUs (Basic Health Units) where specialists are not available. This project will equip BHUs with digital health services so that remote doctors can see live vitals and critical alerts. Machine learning-based alerts will be generated locally for caretakers to rescue patients on time.

The architecture proposed in the SAFE-RH project can be seen in Fig. 1. At the top-level layer of this architecture, cloud or Virtual Private Server (VPS) is used to deploy a web application which is responsible for managing the Electronic Health Record (EHR) of patients, doctor appointments and disease diagnosis based on the EHR. The symptoms and health vitals of the current patient collected from the sensors attached to the patient are also stored. The second layer consists of multiple high performance servers which are responsible for collecting labeled data from the cloud and devices from the fog layer to train machine learning models using different machine learning algorithms.

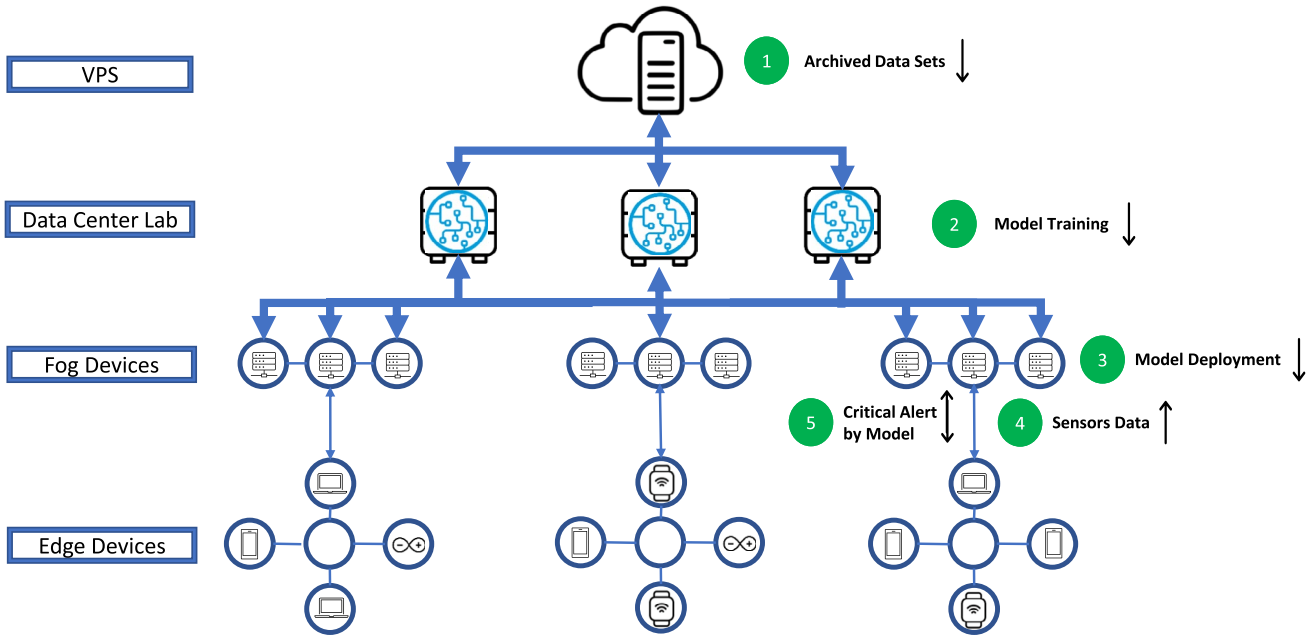


Fig. 1 Proposed architecture and model deployment

The third layer, a combination of several gateways (made up of Raspberry Pi, PCs, etc.), is considered as fog, used to deploy machine learning models for prediction and collection of health sensors reading attached with patients. Sensor data is processed locally through machine learning models for health-related alerts. The last layer known as edge layer contains several health devices such as smart watches, mobile phones, digital health devices such as blood pressure (BP) operating, oxygen metre, electrocardiogram (ECG), glucose metre, etc. The devices collect health readings from patients and send them to a fog layer for further processing. The project goal is to develop a self-predictability and efficient responsive system with ML trained models over fog nodes. However, ML models deployed over fog nodes may cause degradation of the overall system due to concept drift over several nodes. These issues are still an open area in the literature [13] to work on.

Existing research on drift detection in ML models ([10, 14–17]) relies on large datasets, causing delayed drift detection. Moreover, fog and edge computing architectures, where ML models run on many devices, lack reliable mechanisms to update models when drift occurs on single or multiple nodes. We propose an ensemble-based methodology that integrates clustering and classification to enable real-time detection of concept drift in fog computing environments. An observed increase in the dissimilarity or prediction error between the labels inferred by the clustering component and those produced by the classifier serves as an indicator of drift in the classifier's decision function.

In the literature, concept drift is commonly defined as a substantial temporal change in the statistical properties of the feature space, which leads to a degradation in model performance. One common way to monitor such degradation in existing approaches is by tracking the model error rate over time. Let A_0 denote the initial accuracy, which can be defined as the proportion of correctly classified samples during the training or validation phase. Accordingly, the initial error rate E_0 is computed as shown in Eq. 1, where E_0 represents the initial error rate and A_0 represents the initial accuracy.

$$E_0 = 1 - A_0 \tag{1}$$

Similarly, let E_t represent the error rate observed at time t in the newly incoming data computed as $1 - A_t$ where A_t represents the accuracy observed at time t. The drift can then be detected by comparing the current error rate E_t with the initial error rate E_0. If the difference between them exceeds a predefined threshold δ, concept drift is indicated, as shown in Eq. 3. Here, δ represents the drift detection threshold. The value of δ is usually defined differently by different drift detection approaches in literature, depending on their underlying detection mechanism.

$$E_t = 1 - A_t \tag{2}$$

$$D_t = (|E_t - E_0| > \delta) \tag{3}$$

where D_t is a Boolean drift decision that returns True if the difference between the current error E_t and the reference error E_0 exceeds the threshold δ; otherwise, it returns False.

In this study, E_0 is initialized once from validation data used during the initial training phase. It represents the baseline no-drift error rate of the trained model. The current error E_t is computed on the most recent fixed-size detection window. We use fixed-size, non-overlapping windows rather than sliding windows in the reported experiments. After an alarm, E_0 is not reset because the classifier is not retrained during the experiment; the alarm is recorded and the next fixed window is processed. If a deployment scenario includes model retraining after a confirmed drift, then E_0 should be reinitialized using the new post-update validation window.

The above equations describe the generic error-rate-based drift criterion from the literature. In contrast, the proposed Real-time Drift Detection Method (RTDDM) rule does not use E_0 during runtime because true labels are not assumed to be available. Instead, RTDDM computes the classifier–cluster disagreement rate E on each fixed-size window and compares it with the proposed threshold in Eq. 25.

The proposed model drift detection method used an ensemble of supervised with unsupervised learning paradigms to continuously monitor data drift. In this framework, a classifier is trained on an initial labeled reference dataset, while a clustering model is concurrently fitted in the same feature space to produce cluster assignments that can be associated with class labels. This coupling of establishes a collaborative relationship between the two components. RTDDM evaluates an error rate by comparing the classifier's predicted label for an unlabeled new instance with the label estimated from its cluster membership. Initially, both the classifier and the clustering are calibrated on the same reference section of a dataset which ensures a meaningful evaluation to assess changes in error rate over the time.

We validated our approach on synthetic datasets, including the Sine dataset [18] and a real-world COVID-19 dataset. Further methodological and experimental details are explained in upcoming sections. The main research outcomes of this study are:

1. A Real-time Drift Detection Method, named RTDDM (Fig. 2), is proposed to monitor concept drift during model operation without relying on delayed ground-truth labels.
2. A cross-model association mechanism is introduced by comparing the output of a supervised classifier with the cluster-based label generated from the same incoming data. This helps identify model drift in unlabeled data streams.
3. The proposed approach is designed to be model-independent, which makes it applicable to a wide range of classification algorithms rather than being restricted to a particular machine learning model.

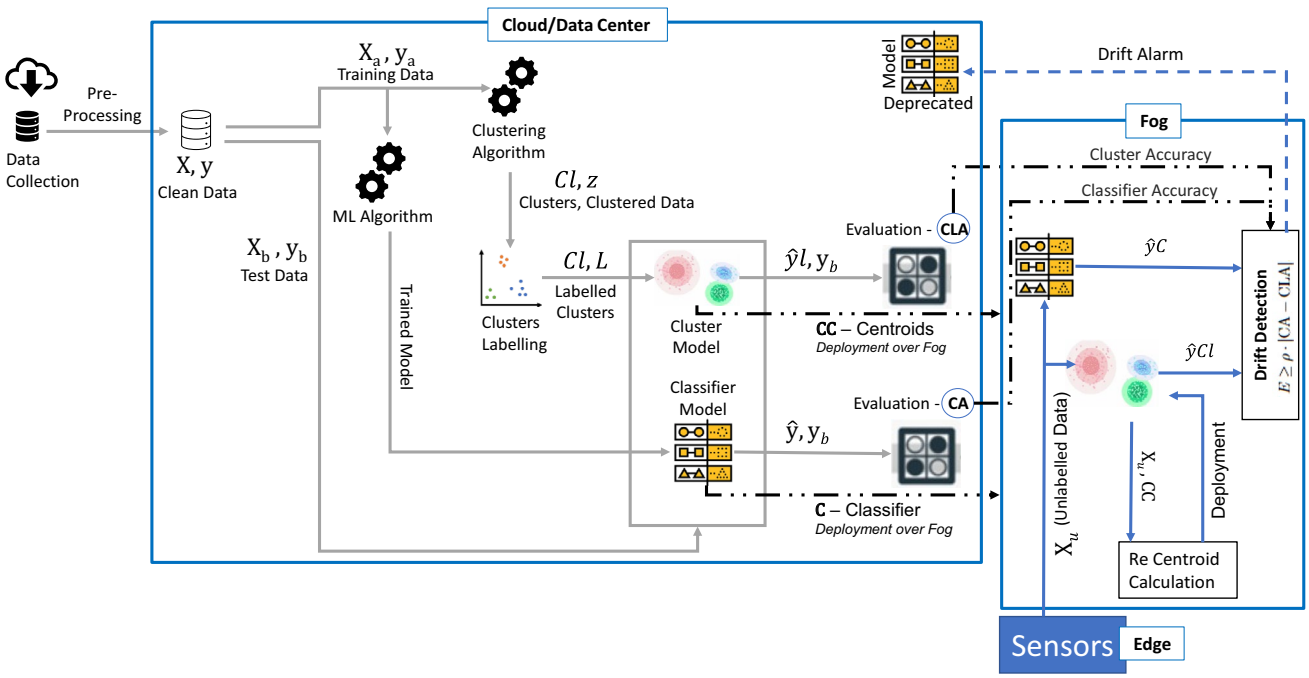


Fig. 2 Proposed RTDDM framework for real-time drift detection. The classifier C, cluster model Cl, centroids CC, classifier accuracy CA, and cluster accuracy CLA are prepared at the cloud/data center and deployed over the fog layer. During deployment, incoming unlabeled data are evaluated by both the classifier and clustering branches, and drift is detected from their disagreement

Algorithm 1 Assign cluster labels by majority vote

Input: $\mathbf{z}$ — cluster indices $(0, \ldots, K-1)$
Input: $\mathbf{y}$ — ground-truth labels (0 or 1)
Output: L — list of K labels, one per cluster
$L \leftarrow [\,]$
for each distinct cluster ID k in $\mathbf{z}$ **do**
    $count_0 \leftarrow 0, \;\; count_1 \leftarrow 0$
    for each sample index i with $z_i = k$ **do**
        if $y_i = 0$ **then**
            $count_0 \leftarrow count_0 + 1$
        else
            $count_1 \leftarrow count_1 + 1$
        end if
    end for
    if $count_0 > count_1$ **then**
        Append 0 to L
    else
        Append 1 to L
    end if
end for
return L

Algorithm 1 Assign cluster labels by majority vote

2 Background and literature review

ML drift, such as sudden, incremental, gradual, or recurrence is discussed in many previously discussed approaches [19]. In fog-based architectures [20], sensors embedded in mobile phones, smart watches, microprocessors, etc generate data that are used for ML predictions. Faults (improper calibration, low battery levels, voltage changes) in sensors can generate readings, which can eventually give incorrect ML predictions, which is known as abnormal drift in the previous research [21]. Various drift detection strategies have been proposed to improve model performance; however, the majority of drift detection methods do not consider abnormal drift.

A recent survey [10] on drift detection groups methods into three categories: error rate-based, data distribution-based, and multiple hypothesis test-based. Several drift detection methods are based on the model's prediction error. In these approaches, drift is identified by monitoring the proportion of incorrectly classified instances, as defined in Eq. 1.

The Drift Detection Method (DDM) [14] is one of the early and widely used error-rate-based techniques for concept drift detection. It treats the sequence of prediction errors as an indicator of possible change in the data stream. After each labeled instance, DDM updates the current error probability and its associated standard deviation, and compares this value with the best error level observed so far. A moderate increase in the error bound is treated as a warning condition, while a larger increase is considered evidence of drift, as shown in Eq. 5. In this equation, p_i and s_i represent the error rate and its standard deviation at instance i, whereas $p_{\min}$ and $s_{\min}$ denote the lowest error rate and lowest standard deviation recorded during the stream. The original evaluation of DDM showed that adapting the classifier after a detected change can improve predictive performance. Since its decision mechanism reacts to a noticeable rise in classification error, DDM is mainly suitable for detecting sudden or abrupt concept drift and requires true labels to compute the error stream.

$$U_i = p_i + s_i, \qquad T_w = p_{\min} + 2s_{\min}, \qquad T_d = p_{\min} + 3s_{\min} \tag{4}$$

$$\text{DDM State} = \begin{cases} \text{Warning state} & \text{if } T_w \le U_i < T_d, \\ \text{Confirmed drift} & \text{if } U_i \ge T_d, \\ \text{Stable state} & \text{otherwise.} \end{cases} \tag{5}$$

The Early Drift Detection Method (EDDM) [22], an extension of DDM, aims to detect gradual changes in data streams earlier. Rather than using only the current classification error, EDDM monitors the distance between successive prediction errors. A stable classifier produces less frequent errors, while decreasing the distances between errors may be an indication of the concept drift. For each instance i, it computes the mean distance between errors p_i'' and its standard deviation s_i''. These values are compared with the best historical value maintained by the detector, represented by $p''_{\max}$ and $s''_{\max}$. As summarized in Eq. 7, EDDM first enters a warning state when the monitored score falls

below the warning threshold, and it confirms drift when the score drops below the drift threshold. The commonly used parameter values are $\alpha = 0.95$ for warning and $\beta = 0.90$ for drift confirmation. Since EDDM relies on the occurrence of classification errors, it requires true labels during stream monitoring and is generally more responsive to gradual drift than the original DDM.

$$R_i = \frac{p_i'' + 2s_i''}{p''_{\max} + 2s''_{\max}}, \tag{6}$$

$$\text{EDDM State} = \begin{cases} \text{Warning state} & \text{if } \beta \le R_i < \alpha, \\ \text{Confirmed drift} & \text{if } R_i < \beta, \\ \text{Stable state} & \text{otherwise.} \end{cases} \tag{7}$$

The Enhanced Reactive Drift Detection Method (ERDDM) [15] was proposed for stream environments where concept drift and class imbalance may appear together. The method works with a recent sliding window of instances and applies a rebalancing step before updating the ensemble. Unlike RDDM, which mainly reacts to changes in the average error rate and may therefore require a larger accumulation of mistakes before raising an alarm, ERDDM incorporates the Page–Hinkley (PH) test to identify changes earlier. The PH statistic compares the cumulative deviation of the observed sequence from its expected behaviour. In Eq. 9, m_t denote the cumulative deviation at time t, while M_t save the smallest value reached by this statistic so far. A drift alarm is raised whenever the difference between these two values crosses the threshold λ which is defined by user. The reported results show that ERDDM can respond to gradual changes earlier than RDDM, particularly in unbalanced data-stream settings.

$$G_t = M_t - m_t \tag{8}$$

$$\text{ERDDM State} = \begin{cases} \text{Confirmed drift} & \text{if } G_t \ge \lambda, \\ \text{Stable state} & \text{otherwise.} \end{cases} \tag{9}$$

The Recurrent Adaptive Classifier Ensemble (RACE) [23] handles recurring concept drift by combining drift detection with model reuse. It uses EDDM [22] to monitor the distance between successive classification errors, making it useful for gradual drift scenarios. Instead of discarding previously learnt classifiers, RACE maintains a repository of models associated with earlier concepts. Whenever a drift is detected, the ensemble can reuse and adapt the relevant archived models for the new data distribution. This strategy reduces the need to train a completely new model after every detected change and reduces the overall computational cost.

The adaptive incremental–decremental SVM (AIDSVM) method [24] made a good utilization of ADWIN [25] as its drift detection component. ADWIN monitors the prediction-error stream through an adaptive window whose size changes according to the stability of the data. To test whether a change has occurred, the window is divided into two sub-windows, and the mean errors of these two parts are compared statistically. Let μ_0 and μ_1 denote the mean errors in the two sub-windows. As expressed in Eq. 11, drift is declared when the absolute difference between these means is at least the adjusted cut value $\epsilon_{\text{cut_adjusted}}$. This cut value is computed from the Hoeffding-based bound in Eq. 12, where m represents two sub-window sizes harmonic mean, σ_W^2 denotes current window variance, and δ'' denotes confidence level used when multiple comparisons are performed.

$$\Delta_\mu = |\mu_0 - \mu_1| \tag{10}$$

$$\text{ADWIN State} = \begin{cases} \text{Confirmed drift} & \text{if } \Delta_\mu \ge \epsilon_{\text{cut_adjusted}}, \\ \text{Stable state} & \text{otherwise.} \end{cases} \tag{11}$$

$$\epsilon_{\text{cut_adjusted}} = \sqrt{\frac{2\sigma_W^2}{m} \ln\left(\frac{2}{\delta''}\right)} + \frac{2}{3m} \ln\left(\frac{2}{\delta''}\right) \tag{12}$$

The methods discussed earlier mainly rely on changes in classification error to identify abrupt or gradual drift. Another group of detectors, however, examines the change in the input distribution itself. A-detection [17] belongs to this category and was proposed for runtime reliability monitoring in Mobile Edge Computing (MEC) systems. The method applies Fractional Distribution Change (FDC) to compare a reference distribution, denoted by P_{old}, with the recent distribution, denoted by P_{new}. This comparison is performed using Jensen–Shannon (JS) divergence, which is derived from the Kullback–Leibler (KL) divergence:

$$\text{KL}(P\|Q) = \sum_i P(i) \log\left(\frac{P(i)}{Q(i)}\right). \tag{13}$$

Unlike KL divergence, JS divergence gives a symmetric comparison between two probability distributions. Let $M = \frac{1}{2}(P_{\text{old}} + P_{\text{new}})$. The JS divergence between the historical and current distributions can be written as:

$$D_{\text{JS}}(P_{\text{old}}, P_{\text{new}}) = \frac{1}{2}\,\text{KL}(P_{\text{old}}\|M) + \frac{1}{2}\,\text{KL}(P_{\text{new}}\|M). \tag{14}$$

A-detection reports drift when the distributional distance reaches or exceeds a predefined threshold λ, as shown in Eq. 15.

$$\text{A-detection State} = \begin{cases} \text{Confirmed drift} & \text{if } D_{\text{JS}}(P_{\text{old}}, P_{\text{new}}) \ge \lambda, \\ \text{Stable state} & \text{otherwise.} \end{cases} \tag{15}$$

The higher value of D_{JS} shows that the recent data distribution has shifted away from the reference distribution.

The threshold λ therefore controls responsiveness of a drift detection. A smaller value leads the method reactive to slight distributional changes whereas on the other-hand a larger value restricts alarms to stronger shifts. The reported results show that A-detection performs better in terms of recall, precision, and F1-score than Earth Mover Distance Detection (EMDD) [26], Euclidean Distance-based Detection (EDD) [27], and Chebyshev Distance-based Detection (CDD) [27].

An Autoencoder-based Drift Detection Method (AEDDM) [28] detects drift by observing how well an autoencoder reconstructs input data. When the stream remains consistent with the learned distribution of the data, the reconstruction loss is expected to stay low. The significant increase in this loss may indicate that the incoming samples no longer follow the same pattern as the data used to train the autoencoder. The AEDDM is very useful for high-dimensional streams, where direct comparison of feature distributions may be hard and where it is important to separate meaningful drift from harmless variation.

For an input sample x_i and its reconstructed version x_i'' the reconstruction loss is computed as:

$$L(x_i, x_i'') = \frac{1}{d}\sum_{r=1}^{d}\left(x_{i,r} - x_{i,r}''\right)^2, \tag{16}$$

where d is the number of input dimensions. AEDDM then evaluates the batch level behaviour of reconstruction errors. In Eq. 18, μ_b and σ_b denote the mean and standard deviation of the batch errors, k controls the sensitivity of the threshold, and *Count* represents the number of error cases observed in the batch.

$$T_b = \mu_b + k\sigma_b \tag{17}$$

$$\text{AEDDM State} = \begin{cases} \text{Confirmed drift}, & \text{if Batch Error} \geq T_b \\ & \text{and } \mathit{Count} \geq \text{Count Threshold} \\ \text{Stable state}, & \text{otherwise} \end{cases} \tag{18}$$

LSTM-based Drift Detection (LSTMDD) [29] is designed for streams where temporal dependency is important. Instead of treating observations as independent instances, the method uses Long Short-Term Memory (LSTM) to learn temporal patterns in the data. At each time step, the observed value y_{actual} is compared with the predicted value $y_{\text{predicted}}$. The difference between these two values forms the prediction residual:

$$e(t) = \left|y_{\text{actual}} - y_{\text{predicted}}\right|. \tag{19}$$

Drift is reported when this residual reaches or exceeds the time-dependent threshold $\lambda(t)$. Since the threshold is updated dynamically, LSTMDD can adapt to changing temporal behaviour and is suitable for real-time monitoring scenarios.

$$\text{LSTMDD State} = \begin{cases} \text{Confirmed drift} & \text{if } e(t) \geq \lambda(t), \\ \text{Stable state} & \text{otherwise.} \end{cases} \tag{20}$$

The major portion of the literature focuses on drift detection algorithms, but in the context of underlying architecture of SAFE-RH, study of fog under health monitoring is also a concern. Due to the features of the fog, the computing tasks in health monitoring application can make significant improvements [30]. Many fog-based health applications have been proposed in the literature that are implemented to achieve the quality response and performance for medical conditions like stroke prediction and fall detection.

A smart e-Health gateway [31] is proposed on a fog computing-based remote health monitoring architecture that uses gateway resources to improve processing efficiency of health sensors' data. The author showed a Remote Health Monitoring System (RHMS) with Early Warning Scores (EWS) using hierarchical fog-assisted cloud computing. The results show improvement in sensing-to-actuation latency in the fog computing up to 140ms over cloud computing.

An ECG Arrhythmia Analysis approach [13] is proposed in which the machine learning model is trained over a cloud and deployed over fog node. In comparison with previous approach [31], this approach uses ML model instead of EWS to capture the patient health state. However, in this approach the issue of concept drift may arise as discussed earlier.

Fog Computing-Based Monitoring System for Cardiovascular Patients [32], used LoRa communication protocol which sends the ECG signal to a Fog layer. The fog layer is deployed with AI algorithm based on deep learning for the detection of Atrial Fibrillation and other heart rhythms. This automation system helps physicians for better diagnosis and achieve better clinical vision that improves therapeutic decision making. However, this system is also lagging with concept drift issue.

Another approach for Severity Prediction of COVID-19 patients [33] in healthcare is Smart Healthcare System that uses IoT sensors in Fog computing for critical tasks processing. This approach, suggests a fusion method that can determine the optimal blend of COVID-19 patient attributes, including demographics, chronic illnesses, symptoms, and laboratory results, to identify the most significant feature set with the greatest predictive capability. But this approach is not capable of handling drifts in the data as well.

XLAAM [34], is a comprehensible LSTM-oriented system designed for categorizing everyday activities and identifying irregularities in a fog-improved intelligent home setting. Sensor data from the smart home is transmitted to fog nodes, where the processes of classification and anomaly detection occur. If any unusual activity is detected, an alert is triggered or a message is dispatched to a healthcare professional or a family member. However, this health related application is not considering concept drift issue.

Foghealth [35] architecture for local processing of heart disease prediction in a person. It uses ensemble approach to pick best prediction among multiple ML models. However, they did not consider the drifting issue in the model prediction which could lead foghealth system to wrong predictions. They used Cleveland database for heart disease prediction.

Ambient Healthcare Approach [36] presents a novel method for enhancing healthcare services through an ambient environment. This approach combines the Hybrid Whale Optimization Algorithm (HWOA) and the Naive Bayes classifier to create a more efficient and effective system for managing health-related data and decision making. This fog-based approach improves accuracy and computational cost for healthcare; however, it assumes that the data distribution does not change.

Recent federated streaming work also highlights that drift can be induced by participation dynamics and non-IID client composition, even when no single synchronized change point exists. Chen et al. describe virtual drift detection and adaptation in federated data stream learning [37], focusing on how drift manifests through the federated process rather than through a single centralized stream.

Federated networked systems are commonly deployed over the fog to support distributed ML models [16]. In this typical setup, a global model is shared across fog nodes and updated through the models that were trained over Fog as well. However, this process often depends on labeled data at each node, which may not be available in many real-time applications. SemiFL [38] addresses this limitation by using unlabeled client data to improve the global model.Instead of relying on clustering-based approaches, it employs self-supervised learning techniques, like contrastive learning, to learn meaningful embedded representations from unlabeled data.

Ensemble and Continual Federated Learning (ECFL) [39], a distributed learning method which detects concept drift across mobile devices. ECFL trains multiple local models and aggregates them globally to improve prediction. For drift detection, it stores predicted labels, confidence scores, and instance values in a sliding window, after which it compares consecutive windows on each edge device. A drift alarm is triggered when the magnitude of change exceeds a predefined threshold. This methodology eventually relies on the model predictive confidence, consequently, if the model is poorly calibrated, it may produce similar confidence scores even in the presence of drift, thereby diminishing the reliability of a drift detection.

In distributed environments, concept drift may appear at different times and with different magnitudes among clients [40]. Some methods detect local model loss at each client, and then affected clients are grouped into drift-based clusters for retraining. This will help to maintain the overall accuracy of the federated application but the way model loss is measured and interpreted remains an important issue.

Concept drift in the federated learning can affect the system in different ways [41]. If drift occurs only on a small subset of devices then affected clients may experience performance degradation, however, if the global model remains mostly stable. In the case if a drift appears across many devices, the global model may also lose accuracy because it cannot adapt quickly enough to the changing data. Drift can also increase the performance variation across devices, where the same model performs well on some clients but poorly on others due to differences in local data distributions.

Concept drift is a major challenge in machine learning as it changes either the statistical properties of incoming data or the relationship between input features and the target class. Although many drift detection methods have been proposed, most rely on labeled test data or synthetic streams. This just in time availability of true labels limits their use in real-time applications where labels are not immediately available in a time frame. In such cases, the system may depend on expert reviews for true label feedback, which is often slow and impractical due to response delays. In sensitive domains such as healthcare the delayed drift detection (Model Degradation) can be dangerous because model prediction errors, especially false negatives may directly affect decision quality.

This motivates the need for a drift detection method that can work with unlabeled incoming data and reduce the dependence on continuous human verification for timely detection of drift. Moreover, when drift occurs rapidly or across multiple data sources, centralized monitoring may not be sufficient. A fog-based distributed architecture can help by detecting drift closer to the data source and supporting timely response across multiple nodes.

Recent studies show growing interest in drift detection methods for streaming environments where labels are delayed, costly, or unavailable. Lukats et al. surveyed fully unsupervised concept drift detectors on real-world streams, while Assis and Souza proposed ADWIN-U as an adaptive-window method for unlabeled drift detection [45, 46]. In healthcare, drift monitoring is also becoming important because changes in patient populations, devices, and

clinical workflows may degrade deployed models over time [44, 49]. Recent fog/edge healthcare studies further emphasize low-latency local processing and cloud-supported validation for continuous telemedicine monitoring [47].

Ilic et al. [49] proposed a drift-aware edge–cloud healthcare monitoring architecture in which drift is first screened at the edge and then validated in the cloud. The edge tier uses a four-detector ensemble consisting of ADWIN, Page-Hinkley, mean-shift, and variance-ratio monitors. A local drift alert is generated when at least two detectors agree within a 60-second window, while the cloud validates the alert using confidence, persistence, and cooldown checks before triggering adaptation.

Compared with these studies, proposed approach provides a lightweight fog-side mechanism that uses classifier–cluster disagreement instead of true labels or purely feature-distribution scores. Therefore, proposed approach is complementary to recent unsupervised and edge-healthcare monitoring work because it offers a practical label-free alarm signal near the data source.

Our study concludes that, many of recent remote health monitoring systems (RHMS) over fog architecture are not integrating concept drift approached for detection. In general, concept drift detection algorithms use synthetic datasets for demonstrating their techniques and no real health dataset is used by these concept drift detection algorithms. Table 1 compares representative drift detection methods and healthcare/fog-related applications. The abbreviations used in the classifier column are: DT = Decision Tree, NB = Naive Bayes, NN = Neural Network, KNN = k-Nearest Neighbor, SVM = Support Vector Machine, RF = Random Forest, LR = Logistic Regression, DNN = Deep Neural Network, CNN = Convolutional Neural Network, LSTM = Long Short-Term Memory, AE = Autoencoder, HT = Hoeffding Tree, MLP = Multilayer Perceptron, EWS = Early Warning Score, JS = Jensen–Shannon divergence, SVD = Singular Value Decomposition, TAE = Task-Agnostic Embedding, BBSD = Black-Box Shift Detection, and CGM = Continuous Glucose Monitoring. This study helps establish approach, that can detect model performance degradation for unlabeled data, particularly in healthcare, where outcomes directly affect human life in fog-node deployments.

3 Proposed approach and framework setup

Our proposed Real-time Drift Detection Method (RTDDM) uses clustering as a proxy labeler that runs in parallel with a trained classifier. The basic idea is to compare the label predicted by the classifier with the label derived from the nearest labeled cluster. When the disagreement between these two outputs increases, it provides an indication that the data distribution may have changed. In this way, RTDDM can monitor incoming unlabeled data during deployment without waiting for ground-truth labels or human annotation.

In this study, real-time drift detection refers to online monitoring of incoming data streams while the model is operating. Instead of analyzing the complete dataset after all samples have arrived, RTDDM evaluates each incoming batch or slice as it is received. The drift evidence is computed by comparing the classifier output with the cluster-based label. Since true labels are not required for test data during deployment, the method is suitable for real-time or near-real-time environments where labels are delayed, costly, or unavailable.

A classifier learns a mapping from feature values to class labels using labeled training data. This learned mapping is then used to predict labels for unseen or real-time instances. In contrast, a clustering algorithm groups instances according to similarity in their feature values without using class labels. After the clusters are formed each generated cluster is determined from the dominant class among the labeled samples assigned to it. This creates two parallel labelers: one based on classification and the other based on clustering. RTDDM uses the agreement or disagreement between these two labelers to generate the drift signal.

The proposed method follows three main steps. First, clustering is applied to the labeled training data using only the feature values, and each cluster is labeled through majority voting. This produces a cluster-based labeler that works alongside the classifier trained on the same data. Second, a reference behavior is established by evaluating both labelers on the test partition and computing their baseline performance. Third, during deployment, incoming unlabeled data are passed through both mechanisms. The mismatch between the classifier prediction and the cluster-based prediction is accumulated as the current error rate. If this error rate exceeds the drift threshold, a drift alarm is raised.

Compared with previous work [14, 15, 22], which commonly assumes access to true labels, RTDDM estimates error behavior without requiring labels at runtime. This is important because true labels are often delayed in real systems or may require expert verification. With the help of clustering as a second labeling mechanism, the proposed method supports timely and label-free drift detection with minimal human involvement. The general workflow is shown in the Fig. 2.

The process starts with data collection where data is gathered from different sources and passed through pre-processing to handle missing values, reduce noise, and normalize feature values. The resulting clean data are transferred to the cloud/data center for a model development.

Let the preprocessed reference dataset be denoted by (X, y), where X contains the input attributes and y contains

Table 1 Drift methods vs health applications

Method/Article	ML model used	Arch	Datasets used	Health data?
DDM (2004) [14]	Perceptron, NN, DT	Cloud	ELEC2, SINIRREL1, SINIRREL2, GAUSS, STAGGER, CIRCLES, SINE1, SINE2	No
EDDM (2006) [22]	DT, KNN	Cloud	ELEC2, CIRCLES, GAUSS, SINE1, SINE1G,	No
ADWIN (2007) [25]	NB, DT	Cloud, Edge	ELEC2, Square Wavelets, Triangular Wavelets, Staircase Wavelet	No
Rahmani et al. (2018) [31]	EWS	Cloud, Fog	–	Yes
HealthFog (2020) [35]	DNN	Fog	Cleveland	Yes
Enhance Reactive Drift Detection (2020) [15]	DT	Cloud	Agarwal, SINE	No
Concept Drift Runtime Reliability Detection (2021) [17]	JS, SVD	MEC	ServiceSurvey (QoS data)	No
AIDSVM (2021) [24]	SVM	Cloud, Edge	SINE1, CIRCLES, COVERTYPE	No
Manias et al. (2021) [16]	CNN	Cloud, Fog	MNIST digit	No
1D-CNN ECG Analysis (2021) [13]	CNN	Cloud, Fog	MIT-BIH	Yes
Deep Learning CD-CI Framework (2021) [42]	DNN	Cloud	KDD Cup	No
RACE (2021) [23]	SVM	Cloud	KDD Cup	No
SemiFL (2021) [38]	WRN-28x2	Cloud, Fog	CIFAR10	No
HWOA-NB (2021) [36]	NB	Cloud, Fog	Pima Indians Diabetes	Yes
Hameed Abdulkareem et al. (2022) [33]	RF, LR	Cloud, Fog	–	Yes
CD-SVM IDS (2022) [43]	SVM	Cloud	KDD Cup	No
XLAAM (2022) [34]	LSTM	Cloud, Fog	ARAS	Yes
Casado et al. (2023) [39]	SVM	Edge	WRD	Yes
Jothimurugesan et al. (2023) [40]	ResNet-18	Edge	FMoW	No
AEDDM (2024) [28]	AE	Cloud	ELEC2, Moving RBF, Varying Distributions (VD), NOAA, Covertype, Gaussian, KDDCUP99, Rotating Hyperplane	No
LSTMDD (2024) [29]	LSTM	Cloud	Sine, Hyperplane, Stagger, Google Cloud Usage Trace	No
Kore et al. (2024) [44]	TorchXRay Vision, TAE, BBSD	Cloud	239,235 chest X-rays from 78,542 patients; natural COVID drift and synthetic demographic/pathology shifts	Yes
Lukats et al. Survey (2025) [45]	HT, NB	Cloud	INSECTS variants, Elec2, NOAA, Outdoor, Poker, Power-Supply, Rialto	No
ADWIN-U (2025) [46]	RF	Cloud	Electricity, GasSensor, Insects-A/G/I, LADPU, Outdoor, Rialto, StarLightCurves, Yoga	No
Telemedicine Fog/Edge Survey (2025) [47]	Survey	Fog/Edge	Surveyed telemedicine and fog/edge healthcare studies; no single benchmark dataset	Yes
CIDDM (2026) [48]	NB, HT	Cloud	Multi-class synthetic and semi-synthetic streams	No
Drift-Aware Edge–Cloud Intelligence (2026) [49]	LR	Edge–Cloud	Synthetic physiological streams and real CGM data	Yes
Virtual Federated Drift Detection (2026) [37]	MLP	Federated	Gaussian (2D/8D), Hyperplane, Circle, CoverType, Electricity, GSD, USD/SGD	No

the corresponding class labels. An individual sample record is denoted as (x_i, y_i), in which x_i represents the feature values of the -th instance and y_i denotes its assigned class. The possible class labels are defined as $L = \{l_0, l_1, \ldots, l_{k-1}\}$, where k is the total number of classes. Since the present study focuses on binary classification, $k = 2$, and therefore $L = \{l_0, l_1\}$.

The dataset is divided into two subsets: a training subset (X_a, y_a) and a reference test subset (X_b, y_b). The same partitioning strategy is applied to both the classifier and the clustering components so that their baseline performance is computed over an identical data conditions.

In the classifier branch, a supervised learning model C is fitted using the training subset (X_a, y_a). This training process is expressed as:

$$C \leftarrow \mathrm{ML}(X_a, y_a).$$

After training, C is evaluated in the reference test subset (X_b, y_b). The predicted labels are compared with the actual labels to calculate the classifier accuracy CA, as mentioned in Eq. 21. This value is stored as the reference or baseline accuracy of the classifier and is later used during deployment at the fog nodes layer.

$$CA = \frac{1}{n_b} \sum_{i=1}^{n_b} \mathbb{I}(\hat{y}_i = y_{\mathrm{b}i}) \tag{21}$$

In Eq. 21, n_b denotes the number of samples records in the reference/initial subset, where $\hat{y}_{bi}$ denotes the label predicted by classifier C for -th test instance, y_{bi} is the associated ground-truth label. The function $\mathbb{I}(\cdot)$ returns 1 when the prediction is correct and 0 otherwise.

In the clustering branch, a clustering algorithm is applied to X_{a} to obtain the cluster model and centroids $CC = \{c_1, \ldots, c_K\}$, written as $Cl \leftarrow \mathrm{Clustering}(X_{\mathrm{a}})$. Since clustering does not directly produce class labels, a label map $L : \{1, \ldots, K\} \rightarrow \mathcal{Y}$ is created using majority voting on the labeled training data, as shown in Eq. 22. The clustering accuracy CLA is then calculated on the test partition (X_b, y_b). For each $x \in X_b$, the nearest centroid c_j is identified and its index is converted into a class label using the precomputed label map $\ell(\cdot)$. This gives the cluster-based prediction $\hat{y}_\ell(x) = \ell(h(x))$, where $h(x)$ returns the nearest-centroid index. The predicted cluster label is then compared to y_b to obtain CLA, as shown in Eq. 23.

$$L_k = \begin{cases} 0 & \text{if } \sum_{i \in Cl_k} 1(y_i = 0) > \sum_{i \in Cl_k} 1(y_i = 1) \\ 1 & \text{otherwise} \end{cases} \quad \text{for each } k = 1, \ldots, K \tag{22}$$

$$\mathrm{CLA} = \frac{1}{n_b} \sum_{i=1}^{n_b} \mathbb{I}(\hat{y}l_i = y_{\mathrm{b}i}) \tag{23}$$

Here, The function $\mathbb{I}(\cdot)$ returns 1 when the prediction is correct and 0 otherwise. n_b denotes the number of samples in the reference subset, $\hat{y}l_i$ represents the cluster-based label for the -th record, and $y_{\mathrm{b}i}$ is the corresponding actual label. The clustering model Cl, centroids CC, label map L, and clustering accuracy CLA are retained for deployment.

The Algorithm 1 describes the majority-vote process used to assign labels to clusters. The input consists of the cluster assignment list z and the corresponding actual labels y. The output *cluster_labels* or L stores one label for each cluster, where the index of L represents the cluster number. Since this work considers binary classification, the algorithm counts the number of samples labeled 0 and 1 in each cluster using *count*_0 and *count*_1. The cluster is then assigned the label with the highest count.

The data center produces the main artifacts required for fog deployment: the trained classifier

C, classifier accuracy CA, cluster model Cl, centroids CC , cluster labels L, and clustering accuracy CLA. These artifacts allow the fog node to make predictions and check drift by comparing the classifier output with the nearest-centroid label, without requiring immediate ground truth.

At the fog node, incoming unlabeled data ($\mathbf{X}_u$) from sensors are passed through both the classifier model C and the clustering model Cl/centroids CC. The label $\hat{y}C$ predicted by the classifier and the label $\hat{y}Cl$ predicted by the clustering branch are compared. A mismatch ($\hat{y}C \neq \hat{y}Cl$) is counted as an error. The current error rate E is calculated from these mismatches, as shown in Eq. 24.

$$E = \frac{1}{N} \sum_{j=1}^{N} 1(\hat{y}C_j \neq \hat{y}Cl_j) \tag{24}$$

From a statistical perspective, E represents the sample proportion of classifier–cluster disagreement events in the current detection window. Under stable conditions, the disagreement rate E is expected to remain close to the reference/initial behavior that was observed during validation. A noticeable increase in E suggests that the relationship between the classifier and the clustering model has changed, which may indicate the presence of drift.

The drift judgment is based on the difference between the accuracy of the reference classifier CA and the accuracy of the reference clustering CLA. In Eq. 25, E represents the current disagreement rate, while $\rho > 0$ controls the detector sensitivity. A smaller ρ makes the detector more responsive to drift, while the larger ρ value makes it more conservative and helps reduce false alarms.

$$\text{RTDDM State} = \begin{cases} \text{Confirmed Drift} & \text{if } E \geq \rho \cdot |\mathrm{CA} - \mathrm{CLA}|, \\ \text{Stable State} & \text{otherwise,} \end{cases} \tag{25}$$

In this study with the help of different evaluations, $\rho = 2$ is set as the default setting because it provides a practical balance between timely drift detection and false-alarm control. With this default value, the decision rule becomes:

$$E \geq 2 \cdot |\mathrm{CA} - \mathrm{CLA}|.$$

The generalised form in Eq. 25 still allows ρ to be adjusted according to the target application. For example, a smaller value can be used when early warnings are more important, while a larger value can be selected when avoiding false alarms is the priority.

In the proposed case, gradual drift alarms may be generated slightly later than standard deviation based detection methods [14, 28]. However, approaches based on the standard deviation require continuously updating the statistical parameters upon every new observation j. Moreover, these methods could exhibit increased sensitivity to noise, potentially leading to a higher rate of false-positive drift detection. This trade-off is further discussed in the experimental section.

In contrast to the generic supervised error-rate criterion in Eqs. 1–3, the proposed RTDDM criterion estimates error behavior without using true labels during runtime. Instead of computing error from actual classification accuracy, RTDDM computes a proxy error from the disagreement between the classifier prediction and the cluster-label prediction. Therefore, Eqs. (25) and (24) are not redundant with Eqs. 1–3; rather, they define the proposed operational no-label error-based drift decision rule.

The experiments also show that the cluster-to-label association should not be treated as permanently fixed. The initial association between clusters and class labels is established using Algorithm 1, where ground-truth labels are available. However, for incoming unlabeled data (X_u) , Algorithm 1 cannot be applied because true labels are not available. Therefore, when new data arrive, the clustering model Cl/centroids CC are updated using only X_u, while the previously learned cluster labels are retained or adjusted through centroid-based re-labeling.

To maintain the consistency of the cluster-to-label association, Algorithm 2 compares the ordering of the initial centroid averages with the ordering of the updated centroid averages. If the relative order remains the same, the existing labels L_0 and L_1 are preserved. If the relative order changes, the labels are swapped. This process does not require ground-truth labels for incoming data; instead, it uses centroid-average behavior as a lightweight mechanism to reduce incorrect label assignment after clustering updates.

The centroid average is computed as:

$$\text{avg}(CC_k) = \frac{1}{m}\sum_{a=1}^{m} CC_{k_a} \tag{26}$$

where m represents attributes in the centroid.

The re-labeling rule is defined as:

$$L_0, L_1 = \begin{cases} L_0, L_1 & \text{if } \left(\text{avg}(CC_0^{\text{init}}) < \text{avg}(CC_1^{\text{init}}) \text{ and } \text{avg}(CC_0^{\text{new}}) < \text{avg}(CC_1^{\text{new}})\right) \\ & \text{or } \left(\text{avg}(CC_0^{\text{init}}) > \text{avg}(CC_1^{\text{init}}) \text{ and } \text{avg}(CC_0^{\text{new}}) > \text{avg}(CC_1^{\text{new}})\right) \\ L_1, L_0 & \text{otherwise.} \end{cases} \tag{27}$$

In simple terms, the labels are retained when the relative positions of the two centroid averages remain unchanged after processing new data. If this position changes then labels are swapped. This re-labeling process helps to maintain the reliability of the cluster-based prediction over new data. This improves the ability of RTDDM to detect drift in the deployed classifier C.

4 Datasets

Several real and synthetic datasets have been used in the literature for evaluating concept drift detection methods. A summary of datasets reported in related studies is given in Table 1. To reduce experimental bias, we used both synthetic and real-world datasets. The synthetic datasets cover controlled abrupt and gradual drift scenarios, while the real-world dataset adds a practical healthcare case where data patterns may change over time and true labels may not always be available immediately.

Mixed SINE data streams adopted in this study were produced using the Mixed stream generator [18]. These datasets are based on sequences of classification functions and are designed to simulate different drift patterns. dataset names that are extended with A contain abrupt drift, whereas dataset names that are extended with G contain gradual drift. Each dataset contains four numerical features that are noise free with a balanced binary class label. These properties make the datasets useful for testing how well a drift detection method responds to both sudden and progressive(gradual) changes in the data distribution.

In addition to the synthetic streams, we used a real-world Kaggle COVID-19 dataset, which repackages the patient-level data released by the Dirección General de Epidemiología (DGE) of the Mexican government [50]. The prediction task is binary: identifying whether a patient is COVID-19 positive or not using routine demographic, clinical, and administrative attributes. In the original schema, Boolean fields are coded as 1 =“Yes” and 2 =“No”, while special values such as 97, 98, and 99 indicate missing, unspecified, or non-applicable entries. The dataset includes demographic attributes such as gender and age, clinical classification information, care-related variables such as patient type, USMER, medical unit, intubed, and ICU, and comorbidity indicators such as pneumonia, pregnancy, diabetes, COPD, asthma, immunosuppression, hypertension, cardiovascular disease, chronic renal disease, other disease, obesity, and tobacco use. For drift detection, the clinical classification was converted into a binary label, where classes 1–3 were mapped to positive and the remaining classes were treated as negative or inconclusive.

The raw COVID-19 file contained 1,048,500 rows and 20 columns. After removing 879,819 exact duplicate records, 168,681 unique records remained for modeling. Medical flags were harmonized by converting $1 \rightarrow 1$ and $2 \rightarrow 0$, while special codes such as 97, 98, and 99 were treated as NaN for imputation and monitoring. The SEX attribute was recoded from 1 =female and 2 =male to 0 =female and 1 = male. AGE was converted to a numeric field, and invalid values below 0 or above 110 were set to NaN, while keeping age as a continuous variable. From the cleaned data,

we also derived interpretable features, including comorbidity_count, risk_factors_count, pregnant female, and age_bucket. To avoid admission-time leakage, downstream care variables such as ICU and INTUBED were excluded from the training feature set. The final processed dataset was saved as *Covid-Data.model-ready.imputed.csv*, and the experiments based on this dataset are reported in Sect. 5.3.

5 Experiments and results

In the experiments, we used a Random Forest classifier together with K-Means clustering to evaluate drift detection on synthetic data streams and a real-world health dataset. In this study, we focus on the *mixed _0101 _gradual* dataset, as these streams are commonly used in the literature for drift detection methods based on the concept of benchmarking.

Table 2 summarizes the datasets used in the experiments, including their type, size, drift/reference setting, and evaluation criteria. The synthetic Mixed datasets are publicly available from Harvard Dataverse, while the COVID-19 healthcare dataset is based on the Kaggle dataset derived from Mexican DGE patient-level records.

5.1 Libraries and tools

We perform experiments in Python using the Scikit-learn library, which provides standard tools for machine learning, clustering, and preprocessing. A Random Forest classifier from the `sklearn.ensemble` package is adopted for the classification component. The classifier was configured with 100 decision trees (`n_estimators = 100`) to improve stability and reduce the risk of overfitting by averaging predictions across multiple trees. A fixed random seed (`random_state = 44`) was used to make the results reproducible in repeated runs. The default bootstrap sampling and feature-selection mechanism of Random Forest were retained, allowing individual trees to learn from different data samples and feature subsets. These settings are suitable for the experimental dataset of 41,000 instances, as the ensemble structure helps the model generalize under changing data distributions, including gradual drift.

The clustering branch employed K-Means to identify changes in the data structure that may reflect drift. Since the considered datasets are binary, the algorithm was configured with two clusters, i.e., `n_clusters` = 2. `K-means++` initialization method is applied to obtain stable reference centroids and to improve the convergence behaviour of the clustering process. The maximum number of iterations was set to 300, with a tolerance value of 10^{-4}, so that the algorithm could stop once the centroid movement became sufficiently small. A fixed random seed (`random_state = 0`) was also used for reproducibility. Before applying K-Means, the input features were scaled to the range $[0, 1]$ using `MinMaxScaler` from `sklearn.preprocessing`, ensuring that all features contributed consistently to the clustering process.

5.2 Synthetic stream-based drift evaluation

The introduced method is assessed by synthetic *mixed_0101_gradual* data stream. This stream consists of 41,000 instances described by four input attributes and includes three predefined drift locations, as reported in Sect. 4. Its drift pattern is gradual, meaning that the underlying distribution evolves progressively across the stream instead of changing at a single abrupt point. For subset-level analysis, the complete stream was arranged into 100 sequential blocks, with 410 records in each block.

5.2.1 Evaluation 1: baseline random forest under gradual drift

This evaluation establishes the baseline response of a Random Forest (RF) classifier when no adaptation is performed after the initial training stage. The RF model was fitted using only the opening block of 410 records. Once trained, the same model was applied to all later blocks without retraining, parameter adjustment, or any form of incremental update. For every block, the predicted class values were matched with the available ground-truth labels to obtain the classification accuracy. This setup provides a reference view of how the trained classifier behaves as the incoming stream gradually departs from the distribution seen during training.

Table 2 Summary of datasets and evaluation criteria

Dataset	Type	Size/features	Drift/reference setting	Evaluation criteria
mixed_0101_ gradual	Synthetic	41000 records; 4 features and 1 class label	Gradual drift starts at 9500, 20000, and 30500.	Detected alarms, matched drift events, first matched alarms, average absolute delay, and missed drifts.
mixed_0101_ abrupt	Synthetic	40000 records; 4 features and 1 class label	Abrupt drift starts at 10000, 20000, and 30000.	Drift alarm behavior and detection response around known abrupt drift points.
COVID-19 healthcare dataset	Real-world healthcare	1048500 raw records and 20 columns; 168681 unique records after removing 879819 duplicates	No artificial drift was injected. Label-based performance-degradation windows were used only as offline reference points.	Classifier performance drop, RTDDM alarm location, and label-free runtime detection using classifier–cluster disagreement.

Figure 3 reports the baseline RF performance across the stream blocks. The horizontal axis shows classification accuracy in the range of 0 to 1, while the vertical axis indicates the subset index. Each plotted point shows the accuracy achieved for a specific block, and the connected curve summarizes the overall change in performance throughout the stream.

The model performs well in the earlier subsets because the data is still close to the initial training distribution. However, after the early stage, the accuracy gradually decreases, showing that the classifier becomes less reliable as the data distribution changes. This confirms that a model trained only once may not remain stable under gradual drift. In this experiment, true labels were used only to evaluate accuracy; later experiments focus on drift detection without relying on labels during deployment.

5.2.2 Evaluation 2: random forest with model adaptation

The second evaluation examines whether periodic model adaptation can reduce the impact of gradual drift on the Random Forest classifier. Similarly to the baseline setting, the classifier was trained for the first time in the initial subset containing 410 records. Unlike the fixed baseline (red), this setting refreshed the classifier after each subset using the available labelled records (Blue plotted line). This allowed the classifier to incorporate recent data patterns and adjust to the changing distribution, rather than depending only on the original training subset.

Figure 4 compares the non-adaptive and adaptive classifiers. The red line represents the baseline model from Experiment 1, while the blue line represents the retrained model. The X-axis shows accuracy, and the Y-axis shows the subset index. In the early subsets, both models show similar behavior because the distribution has not changed significantly. As gradual drift becomes more visible, the adaptive model maintains better accuracy than the fixed model. The result shows that retraining can help the classifier follow the changing distribution, although this setting requires true labels for adaptation.

5.2.3 Evaluation 3: fixed means clustering under gradual drift

In this setting, K-Means is treated as a fixed clustering baseline for cluster-based prediction under gradual drift. The model is fitted only once on the opening subset of the stream. The clusters obtained from this subset are then used as reference groups. For each cluster, the assigned class is selected by identifying the class that appears most frequently among the labelled records belonging to that cluster. Following this initial labeling step, the induced cluster–label association was held fixed and subsequently used to predict labels for all later subsets. For any new instance, the predicted label was derived from the class label associated with the nearest cluster centroid. Clustering-based classification accuracy was then computed by comparing these predicted labels with the corresponding ground-truth labels.

Figure 5 presents the fixed K-Means clustering performance setup. The plot shows that static clustering also loses accuracy as the stream gradually changes. Since there is no update over original cluster structure or label mapping,

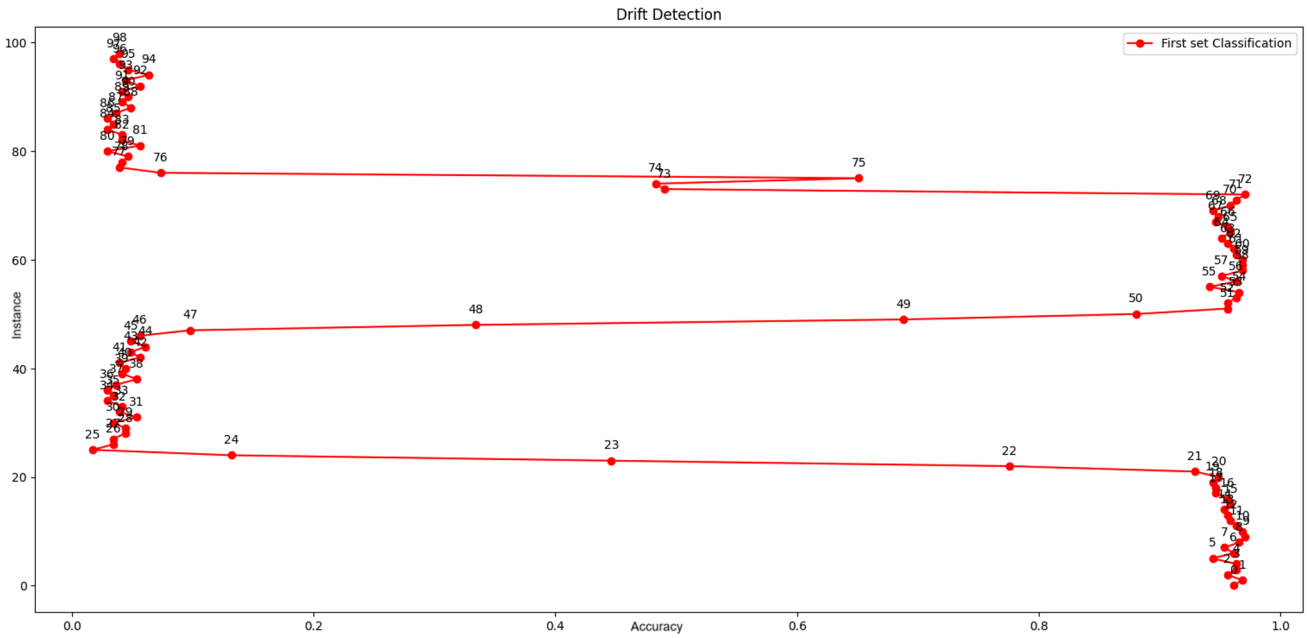


Fig. 3 Baseline random forest performance under gradual drift

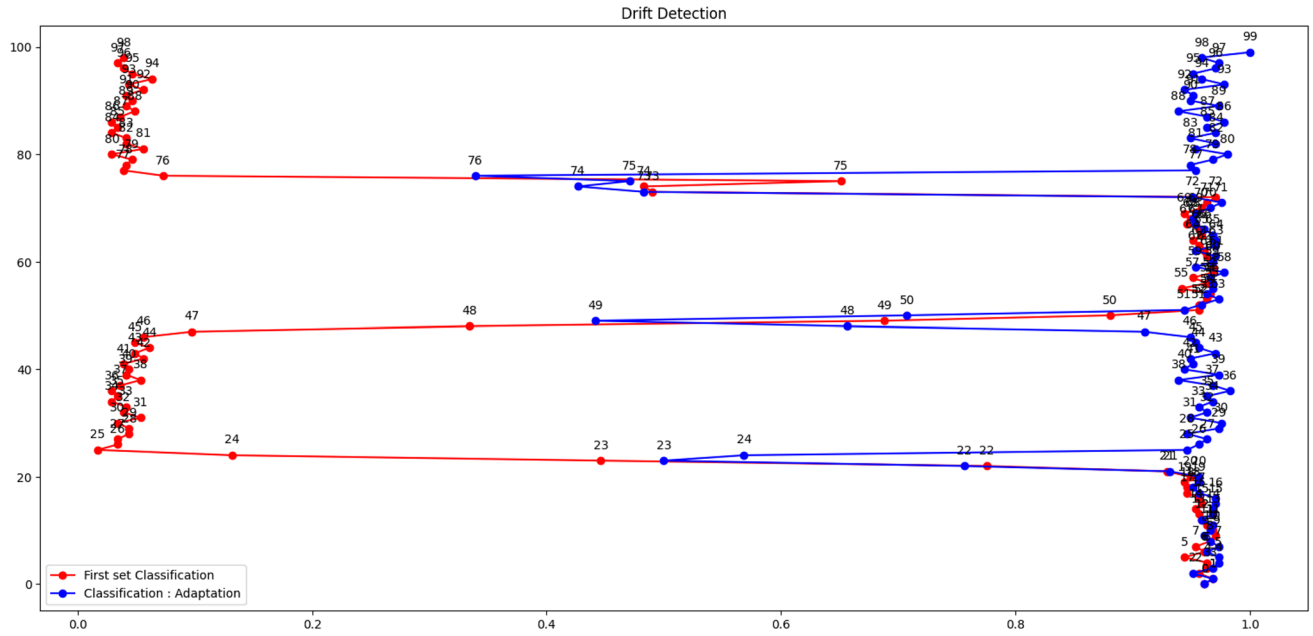


Fig. 4 Random forest performance with and without model adaptation. The red line shows the non-adaptive classifier, while the blue line shows the adaptive classifier

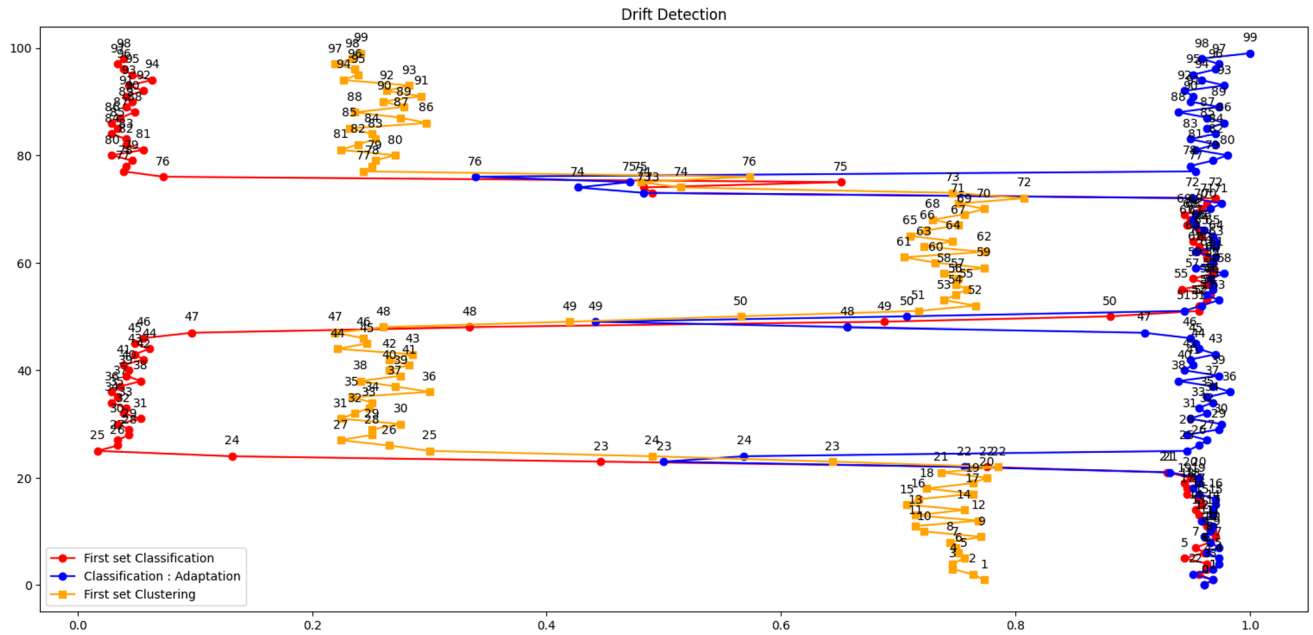


Fig. 5 Static K-Means clustering under gradual drift. The orange line shows clustering-based prediction accuracy compared with the random forest results

the clustering model gradually becomes less aligned with the later data subsets. This behavior highlights the need for adapting the clustering structure when gradual drift changes the data distribution.

5.2.4 Evaluation 4: adaptive K-means clustering with true labels

In this evaluation, K-Means clustering was reapplied to each incoming subset so that the cluster structure could adapt to gradual changes in the data distribution. After clustering each subset, cluster labels were assigned using the majority

class of records in that subset. If $Cl_k^{(t)}$ represents the k-th cluster in subset t, its label was assigned as:

$$L_k^{(t)} = \text{majority}(y_i \mid i \in Cl_k^{(t)}).$$

Each new record was then assigned the label of its nearest cluster. The clustering accuracy for subset t was calculated as:

$$\text{Accuracy}^{(t)} = \frac{1}{n}\sum_{j=1}^{n} \mathbb{I}(\hat{y}_j^{(t)} = y_j),$$

where $\mathbb{I}(\cdot)$ returns 1 when the predicted label matches the true label and 0 otherwise.

Figure 6 shows the adaptive clustering result, compared to the static clustering curve, adaptive clustering gives a more stable trend because the clusters are updated for each new subset. However, this experiment still uses true labels to assign cluster identities after each re-clustering step. Therefore, it mainly serves as an upper-bound comparison to show how clustering could perform when label information is available.

5.2.5 Evaluation 5: adaptive clustering without true labels

This evaluation assesses the proposed adaptive clustering process without relying on true labels during reclustering. K-Means is reapplied to each incoming subset to update the cluster structure, while the cluster-label mapping is maintained from the earlier labelled reference data. This setting represents the unlabeled monitoring condition considered in the proposed drift detection framework. The clusters were labeled only once using the first subset. For the later subsets, the cluster labels were adjusted using the re-labeling strategy described in Algorithm 2. Instead of checking true labels, the method compares the average centroid values of the new clusters, such as avg(CC_0^{new}) and avg(CC_1^{new}), and retains or swaps the cluster labels accordingly. This keeps the process label-free after initialization while still allowing the cluster identities to adapt as the data changes.

Figure 7 compares this label-free adaptation with adaptive clustering that uses true labels. The subfigures show the overall trend, the comparison between static and adaptive clustering, and a closer view of the drift-affected region. The gray and black curves follow a similar pattern, which suggests that centroid-based re-labeling can approximate the behavior of label-based adaptation. This is important for practical stream settings, where true labels are often delayed, costly, or unavailable.

5.2.6 Evaluation 6: drift indication from label-free clustering adaptation

This evaluation examines whether label-free adaptive clustering can be used as an indicator of drift. The purpose is not only to report clustering accuracy, but also to analyse whether the disagreement between the initial Random Forest predictions and the adapted clustering output reflects gradual changes in the data stream. In Fig. 8, the brown line with cross markers shows the comparison between the initial Random Forest predictions and the label-free clustering

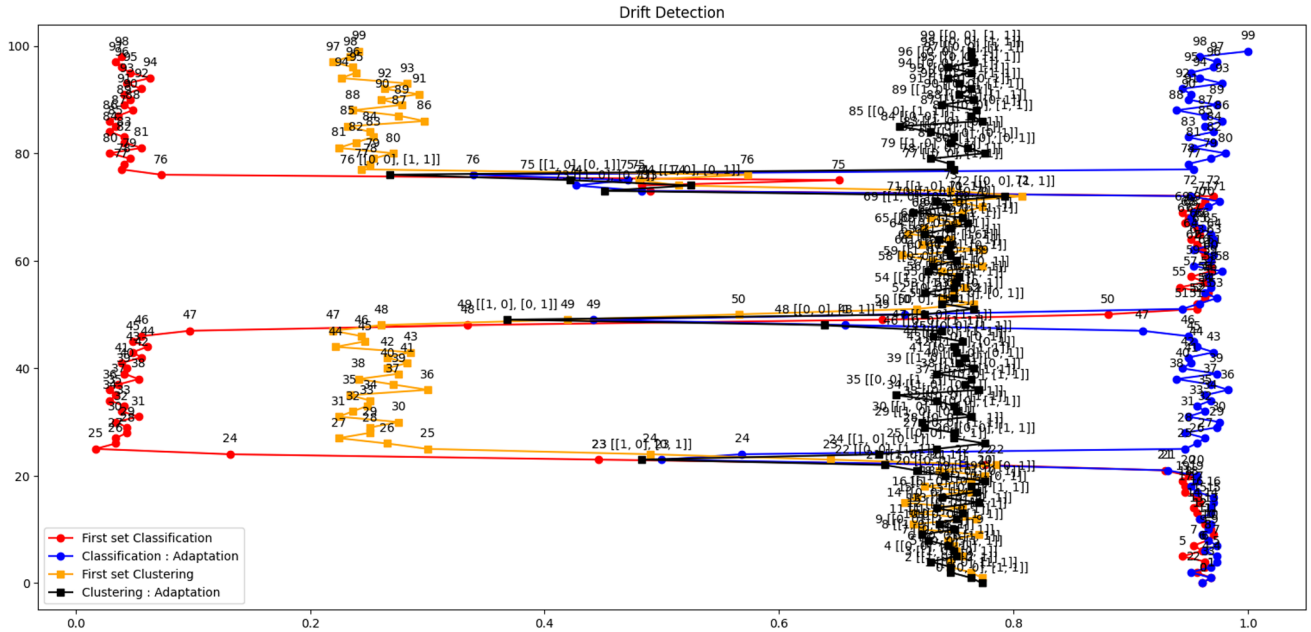
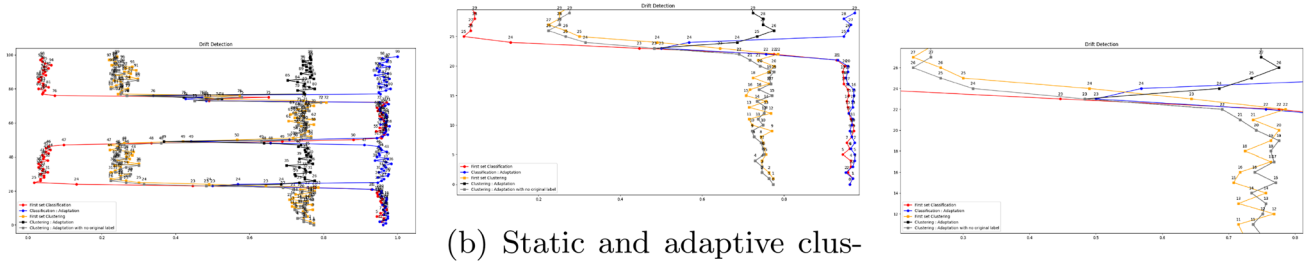
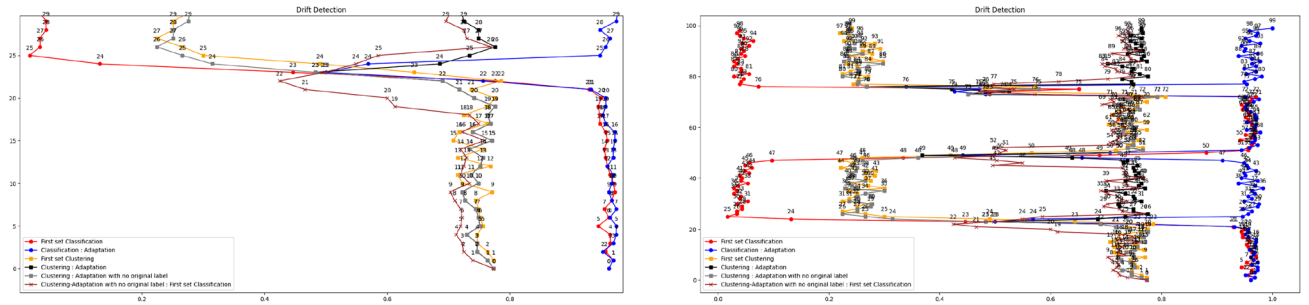


Fig. 6 Adaptive K-means clustering with true labels. The black line shows the accuracy of clustering after re-clustering each subset



(a) Overall accuracy trend (b) Static and adaptive clustering (c) Close-up view

Fig. 7 Adaptive clustering in the absence of ground-truth labels. The gray curve represents unsupervised (label-free) clustering adaptation, whereas the black curve represents clustering adaptation performed with access to the true class labels



(a) Zoomed view for subsets 22–29 (b) Full view across all subsets

Fig. 8 Drift indication using label-free clustering adaptation. The crossed brown line highlights changes around the gradual drift regions

adaptation output, without using true labels during the monitoring process.

The stream contains three gradual drift regions, visible around subsets 22–24, 48–49, and 74–75. In these regions, the plot shows noticeable changes in the behavior of the label-free clustering adaptation. The zoomed view highlights the first drift region, while the full view shows the same pattern across the complete stream. These changes indicate that the proposed label-free clustering process can provide an early signal when the current data no longer follows the initial distribution. This makes the approach useful in scenarios where true labels are not available at detection time, but the system still needs to identify when model adaptation may be required.

5.3 Drift detection in real healthcare dataset

As discussed in previous sects. 4, we used the Kaggle COVID-19 dataset which predict whether a patient has COVID-19 or not (binary: positive vs. negative/inconclusive). However, after pre-processing our first objective was to test that this dataset contains data drifts or not. The low and hight accuracy patterns after certain records in experiments reveal that gradual drift exists at 9 different ranges of points. We will further discuss it in the experiments below.

We stream the imputed, model–ready file `Covid-Data.model-ready.imputed.csv` with $N = 168{,}681$ records as $K = 146$ contiguous sets. As per earlier experiments, each experiment in this section reports three traces per set s: (i) **Classification–Baseline** (red) is the RF accuracy when the model trained on s=1 is applied to s; (ii) **Clustering: Adaptation with no original label** (gray) is the accuracy obtained by reclustering the set with k-means and projecting cluster IDs to labels using the cluster-to-class mapping initialized in the first set and subsequently maintained through centroid-based relabeling, as described in Algorithm 2; (iii) **Cluster label prediction + Baseline classifier label prediction** (brown) reflects the agreement accuracy between the two prediction streams in set s.

In context of Eq. 24, our drift detector uses the *unlabeled* disagreement rate (E) between the two predictors in equation below, where N_s is the size of set s.

$$E_s = \frac{1}{N_s} \sum_{j=1}^{N_s} 1(\hat{y}C_j \neq \hat{y}Cl_j), \tag{28}$$

A drift alarm is raised when the disagreement exceeds defined threshold:

Algorithm 2 Clustering Adaptation

Input: Initial centroids CC^{init}, initial labels L^{init}, new data points xu, number of attributes m in xu
Output: Updated labels L_0, L_1, updated centroids $CC_0^{\text{new}}, CC_1^{\text{new}}$
Step 1: Re-cluster new data points xu to form new clusters Cl_0 and Cl_1
Step 2: Compute the average of the initial centroids:
    $\text{avg}(CC_0^{\text{init}}), \text{avg}(CC_1^{\text{init}})$
Identify the smaller average cluster index:
if $\text{avg}(CC_0^{\text{init}}) < \text{avg}(CC_1^{\text{init}})$ **then**
    smaller_mean_cluster_index $\leftarrow 0$
else
    smaller_mean_cluster_index $\leftarrow 1$
end if
Step 3: Calculate the average of the centroids for each cluster:
for each cluster $k \in \{0, 1\}$ **do**
    Initialize $\text{avg}(CC_k^{\text{new}}) \leftarrow 0$
    for each attribute a in xu (where $a = 1$ to m) **do**
        $\text{avg}(CC_k^{\text{new}}) \leftarrow \text{avg}(CC_k^{\text{new}}) + \frac{CC_{k_a}^{\text{new}}}{m}$
    end for
end for
Step 4: Compare the average centroid values using smaller_mean_cluster_index:
if (smaller_mean_cluster_index $= 0 \wedge \text{avg}(CC_0^{\text{new}}) < \text{avg}(CC_1^{\text{new}})$) **or** (smaller_mean_cluster_index $= 1 \wedge \text{avg}(CC_0^{\text{new}}) > \text{avg}(CC_1^{\text{new}})$) **then**
    Keep labels L_0 for Cl_0 and L_1 for Cl_1
else
    Swap labels: assign L_0 to Cl_1 and L_1 to Cl_0
end if
Step 5: Output: The updated labels L_0, L_1 and the updated centroids $CC_0^{\text{new}}, CC_1^{\text{new}}$

Algorithm 2 Clustering Adaptation

$$\text{Drift}(s) = \begin{cases} \text{true}, & \text{if } E_s \geq 2\,|\text{CA}_s - \text{CLA}_s| \\ \text{false}, & \text{otherwise.} \end{cases} \tag{29}$$

Put simply, when the classifier and the clustering proxy achieve similar accuracy, their accuracy gap $|\text{CA}_s - \text{CLA}_s|$ is small, resulting in a lower threshold, so even a small disagreement rate can trigger a drift alarm. Conversely, if the two predictors already differ a lot, the rule automatically raises the bar, preventing false positives that would appear simply because one signal is noisy. In the figures the brown line shows their *agreement rate*, $1 - E_s$; whenever that brown trace dips, the predictors are clashing more often, driving E_s up toward (or beyond) the drift threshold.

Figures 9–11 illustrate three regimes; we summarize each below and link the visual behaviour to the rule (29).

5.3.1 Experiment 1: early drift

Figure 9 shows an early change at **set 4**. In absolute indices this set spans (3469, 4624) (midpoint $\approx$ 4047); in the test stream (after removing the 1156 training records) it corresponds to indices (2313, 3468). At this set the red RF accuracy bends downward while the gray clustering accuracy remains comparatively smooth. Simultaneously, the brown agreement curve dips, implying E_4 increases. Empirically E_4 crosses the adaptive threshold $2\,|\text{CA}_4 - \text{CLA}_4|$, so Drift(4) = true by (29). This pattern is consistent with a covariate shift that the fixed model (trained only on s=1) fails to absorb, whereas the unsupervised adapter re-estimates structure in the new set.

5.3.2 Experiment 2: stable regime (No Drift)

In Figure 10 the three traces move in parallel with no abrupt separation. The agreement curve (brown) does not exhibit sustained dips; equivalently, the disagreement stays below threshold, $E_s < 2\,|\text{CA}_s - \text{CLA}_s|$ for successive sets. Thus Drift(s) = false throughout this segment, indicating a stationary regime.

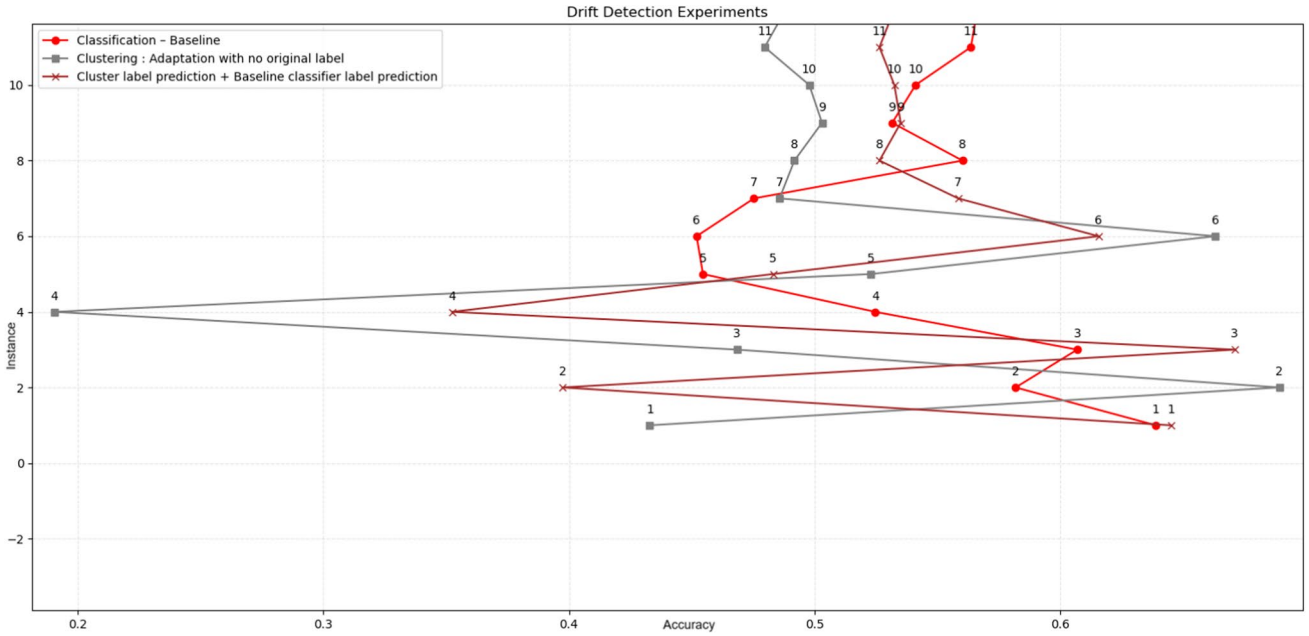


Fig. 9 Experiment 1 (early drift at set 4). Red: RF trained on set 1 and applied to later sets. Gray: k-means adapter with cluster-to-class mapping maintained through centroid-based relabeling. Brown: agreement accuracy between the two predictors ($1-E_s$). The inequality $E_4 \geq 2\,|\mathrm{CA} - \mathrm{CLA}|$ holds, triggering a drift alarm for records 3469–4624

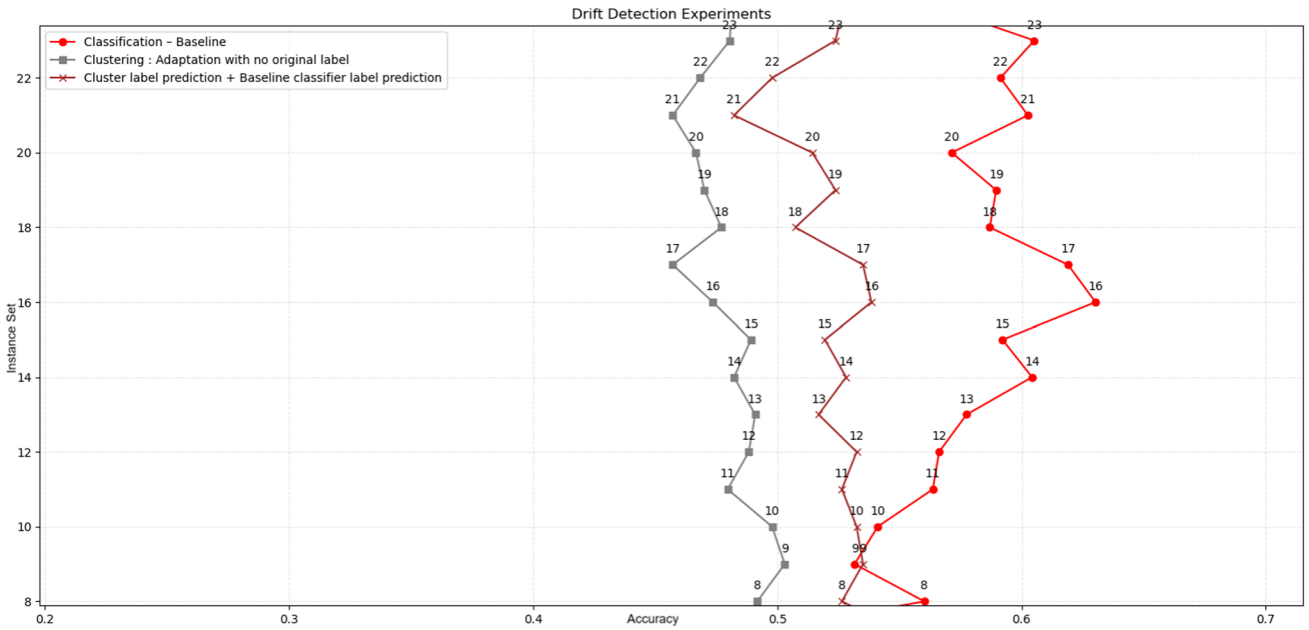


Fig. 10 Experiment 2 (stable regime). The RF baseline (red), clustering adapter (gray), and their agreement (brown) evolve smoothly; the disagreement E_s remains below the adaptive threshold, so no drift is signaled

5.3.3 Experiment 3: later drift

Figure 11 exhibits a second change at set 27. The absolute index range is (30057, 31212) (midpoint $\approx$ 30635). Here the RF baseline diverges while the clustering adapter retains a smoother trajectory; the agreement curve dips again, implying E_{27} rises above $2\,|\mathrm{CA}_{27} - \mathrm{CLA}_{27}|$. Consequently Drift(27) = true by (25). As with Experiment 1, this points to a distributional shift (e.g., altered covariate mixture) that the fixed classifier cannot handle without adaptation.

5.4 Comparison with existing drift detection methods

This section provides a comparative assessment of the proposed RTDDM drift detection approach against three standard stream-based detectors implemented in River, namely DDM, ADWIN, and Page-Hinkley (PH). The assessment was carried out on the *mixed_0101_gradual* stream, where gradual drift is known to occur near instance positions 9,500, 20,000, and 30,500. For the learner, River's `ARFClassifier` was selected as the Adaptive Random Forest implementation. The learner was initialized using the first 1,000 instances of the stream. After this initialization phase, the remaining instances were processed without further training, so that the detector responses could be observed under a fixed-model setting. The baseline detectors were run with the default parameter settings available in River [51].

Figure 12 illustrates how the alarms generated by the selected detectors align with the known drift intervals. The classifier accuracy (blue) trend is included to show how predictive performance changes as the stream evolves. The reported alarm locations are distinguished through detector-specific markers, while dashed vertical lines indicate the corresponding alarm positions. The known drift intervals are shown below the horizontal axis, allowing a direct visual comparison between the expected drift regions and the alarms produced by each method.

During the initial part of the stream, the classifier maintains strong predictive performance because the incoming records still resemble the patterns learned from initial training subset. When the stream approaches the known drift region, the distribution starts changing gradually, which leads to a visible reduction in classifier accuracy. DDM identifies one of the three drift events, whereas ADWIN identifies all three. Page-Hinkley also detects changes, but its response is less consistent in some regions. In comparison, the proposed RTDDM (NO-LABEL) approach reports alarms close to the actual drift locations, approximately around instances 9,430, 20,090, and 30,340. This shows that the proposed method can identify gradual drift without requiring true labels during detection, which is useful in streaming environments where labels are delayed, costly, or unavailable.

Overall, the results indicate that the RTDDM approach provides competitive drift detection performance compared with existing River-based detectors. More importantly, it maintains this behavior under a label-free detection setting, making it suitable for practical applications where continuous access to labeled data cannot be assumed.

Table 3 presents an event-based quantitative comparison of the drift detectors. The known drift locations of the

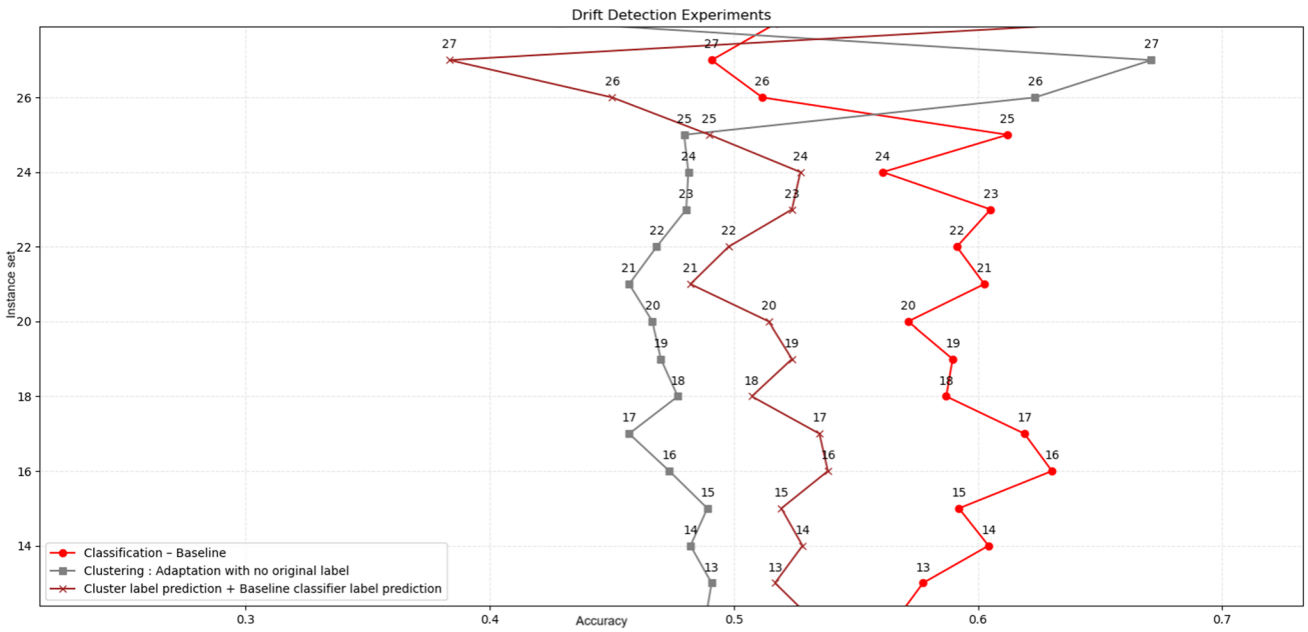


Fig. 11 Experiment 3 (later drift at set 27). A second drift is detected after disagreement exceeds the threshold: $E_{27} \geq 2\,|\mathrm{CA} - \mathrm{CLA}|$. Absolute record span: 30057–31212

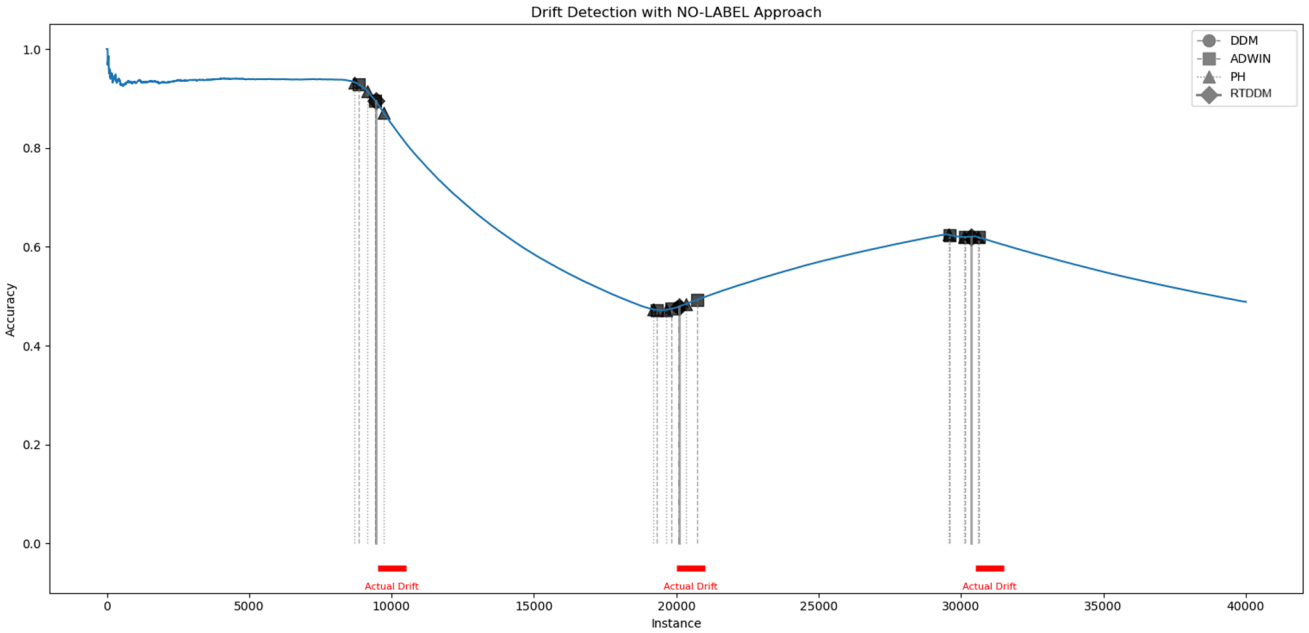


Fig. 12 Comparison of the proposed RTDDM approach with DDM, ADWIN, and Page-Hinkley on the *mixed_0101_gradual* stream. The blue curve shows classifier accuracy over instances, the markers indicate detected drift alarms, and the red horizontal bars represent the known gradual drift regions

Table 3 Event-based comparison of drift detection methods

Method	Total alarms	Matched events	First matched alarms	Avg. absolute delay	Missed drifts
DDM	1	1/3	20705	705.0	2
ADWIN	9	3/3	9216, 19712, 30112	320.0	0
Page-Hinkley	9	3/3	9262, 19771, 30144	274.3	0
RTDDM	3	3/3	9430, 20090, 30340	106.7	0

mixed_0101_gradual stream, i.e., 9500, 20000, and 30500, were used as reference drift points. For each method, the first alarm associated with each known drift event was used to calculate the average absolute detection delay. Extra alarms beyond the matched drift events indicate repeated alarms around the drift regions, while missed drifts indicate known drift events for which no valid alarm was raised.

As shown in Table 3, the proposed RTDDM approach detected all three known drift events with the lowest average absolute delay of 106.7 instances. DDM detected only one of the three drift events, whereas ADWIN and Page-Hinkley detected all three drift events but generated more total alarms. This shows that the proposed method provides a focused alarm pattern around the known drift regions while maintaining label-free runtime operation.

6 Limitations and future work

The current experimental evaluation of the proposed RTDDM framework is limited to binary classification problems. In this study, the clustering component is configured with $K = 2$, and clusters are assigned binary class labels using the initially labeled reference data. Therefore, the present evaluation validates the proposed classifier-cluster disagreement mechanism only in a two-class setting.

Extending RTDDM to multi-class classification is possible in principle, but it introduces additional challenges. In a multi-class setting, the number of clusters may need to be increased according to the number of classes, or even set higher than the number of classes when one class forms multiple sub-groups. This also makes cluster labeling, cluster-label updating, and disagreement-rate calculation more complex. Class imbalance and overlapping class distributions may further affect the stability of cluster-to-class mapping.

As future work, we plan to extend RTDDM to multi-class data streams by redefining cluster purity, cluster-label assignment, and threshold selection for multi-class error/disagreement rates. A practical direction is the elderly well-being monitoring case, where activities such as sitting, standing, walking, and falling represent a multi-class classification problem.

7 Conclusion

This study presented a drift detection framework that combines classification with adaptive clustering. The experiments showed that a classifier trained only on the initial data, as well as fixed clustering, becomes less reliable when the data distribution gradually changes. This confirms the need for a monitoring mechanism that can follow evolving data patterns instead of relying only on the initial training distribution.

To address this issue, the proposed approach re-clusters each incoming data subset and updates cluster labels using centroid-based relabeling. This reduces the dependency on continuous ground-truth labels, which are often delayed, expensive, or unavailable in real-world streaming environments. The experimental results showed that the proposed method can identify drift close to the known drift regions and provide useful early indications of distributional change.

Overall, the proposed framework offers a practical and lightweight solution for drift monitoring in dynamic data streams. Its main strength is that it uses the disagreement between the initial classifier behavior and the adapted clustering output to detect drift without requiring labels during detection. This makes the approach suitable for applications where models are deployed for long periods and the data distribution may change over time. In future work, the framework can be extended to multi-class problems, larger real-world datasets, and distributed environments where drift may occur locally before affecting the global model.

Acknowledgements This research is being carried out under the SAFE-RH project which is funded by European Commission under Erasmus+ programme.

Author contributions Salman Ahmed conceived the study, performed the experiments, and drafted the manuscript. Nayyer Masood supervised the paper, provided overall research guidance and contributed technical expertise to the machine-learning methodology. All authors reviewed and approved the final version.

Data availability All the data analyzed and presented in this study are available from the corresponding author upon reasonable request

Code availability The datasets used in this study are publicly available from their original sources, as cited in Sect. 4. The preprocessing steps, model parameters, experimental settings, and evaluation procedure are described in the manuscript to support reproducibility. The implementation and experimental scripts can be made available from the corresponding author upon reasonable request.

References

1. Drake, C., Lian, T., Cameron, B., Medynskaya, K., Bosworth, H.B., Shah, K.: Understanding telemedicine's "new normal": variations in telemedicine use by specialty line and patient demographics. Telemed. e-Health **28**(1), 51–59 (2022)
2. Anderson, G.F., Steinberg, E.P., Holloway, J., Cantor, J.C.: Paying for hmo care: issues and options in setting capitation rates. Milbank Q. (1986). https://doi.org/10.2307/3349925
3. Gulshan, V., Peng, L., Coram, M., Stumpe, M.C., Wu, D., Narayanaswamy, A., Venugopalan, S., Widner, K., Madams, T., Cuadros, J., et al.: Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs. JAMA **316**(22), 2402–2410 (2016)
4. Sockrider, M., Jamil, S., Santhosh, L., Carlos, W.G.: Covid-19 infection versus influenza (flu) and other respiratory illnesses. Am. J. Respir. Crit. Care Med. **202**(10), 27–28 (2020)
5. Hulsen, T., Jamuar, S.S., Moody, A.R., Karnes, J.H., Varga, O., Hedensted, S., Spreafico, R., Hafler, D.A., McKinney, E.F.: From big data to precision medicine. Front. Med. (2019). https://doi.org/10.3389/fmed.2019.00034
6. Brenner, S.K., Kaushal, R., Grinspan, Z., Joyce, C., Kim, I., Allard, R.J., Delgado, D., Abramson, E.L.: Effects of health information technology on patient outcomes: a systematic review. J. Am. Med. Inform. Assoc. **23**(5), 1016–1036 (2016)
7. Gong, Y., Chai, J., Yang, M., Sun, P., Sun, R., Dong, W., Li, Q., Zhou, D., Yu, F., Wang, Y., et al.: Effects of ambient temperature on the risk of preterm birth in offspring of adolescent mothers in rural henan, china. Environ. Res. **201**, 111545 (2021)
8. Butt, M., Khan, A.R., Abidi, S.T.F., Yasmin, R., Hussain, H., Afsar, A.: Characteristics electrocardiogram changes and their outcome in patients presenting with corona virus disease. Pak J. Med. Health Sci. **14**(4), 1303–1305 (2020)
9. Strack, B., DeShazo, J.P., Gennings, C., Olmo, J.L., Ventura, S., Cios, K.J., Clore, J.N.: Impact of hba1c measurement on hospital readmission rates: analysis of 70,000 clinical database patient records. BioMed research international **2014** (2014)
10. Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., Zhang, G.: Learning under concept drift: A review. IEEE Trans. Knowl. Data Eng. **31**(12), 2346–2363 (2018)
11. Awotunde, J.B., Adeniyi, A.E., Falola, P.B., Srinivasan, K., Chauhan, G.S., Jadon, R.: Chapter 12 - fog computing and internet of things technology for throughput and latency in the smart sensor network. In: Awotunde, J.B., Bhoi, A.K., Barsocchi, P., de Albuquerque, V.H.C. (eds.) Advances in Fog Computing and the Internet of Things for Smart Healthcare, pp. 249–280. Elsevier (2026). https://doi.org/10.1016/B978-0-443-33441-2.00005-7
12. Mouradian, C., Naboulsi, D., Yangui, S., Glitho, R.H., Morrow, M.J., Polakos, P.A.: A comprehensive survey on fog computing: State-of-the-art and research challenges. IEEE Commun. Surv. Tutor. **20**(1), 416–464 (2017)
13. Cheikhrouhou, O., Mahmud, R., Zouari, R., Ibrahim, M., Zaguia, A., Gia, T.N.: One-dimensional cnn approach for ecg arrhythmia analysis in fog-cloud environments. IEEE Access **9**, 103513–103523 (2021)
14. Gama, J., Medas, P., Castillo, G., Rodrigues, P.: Learning with drift detection. Brazilian Symposium on Artificial Intelligence, 286–295 (2004) Springer
15. Toor, A.A., Usman, M., Younas, F., Fong, A.C.M., Khan, S.A., Fong, S.: Mining massive e-health data streams for iomt enabled healthcare systems. Sensors (Basel, Switzerland) **20**(7), 2131 (2020)
16. Manias, D.M., Shaer, I., Yang, L., Shami, A.: Concept drift detection in federated networked systems. In: 2021 IEEE Global Communications Conference (GLOBECOM), pp. 1–6. IEEE (2021)
17. Wang, L., Chen, S., He, Q.: Concept drift-based runtime reliability anomaly detection for edge services adaptation. IEEE Trans. Knowl. Data Eng. (2021). https://doi.org/10.1109/tkde.2021.3127224

18. Lobo, J.L.: Synthetic datasets for concept drift detection purposes. Harvard Dataverse (2020). https://doi.org/10.7910/DVN/5OWRGB
19. Liu, A., Song, Y., Zhang, G., Lu, J.: Regional concept drift detection and density synchronized drift adaptation. In: IJCAI International Joint Conference on Artificial Intelligence, (2017)
20. Karunanithy, K., Velusamy, B.: Edge device based efficient data collection in smart health monitoring system using wireless body area network. Biomed. Signal Process. Control **72**, 103280 (2022)
21. Tsai, F.-K., Chen, C.-C., Chen, T.-F., Lin, T.-J.: Sensor abnormal detection and recovery using machine learning for iot sensing systems. In: 2019 IEEE 6th International Conference on Industrial Engineering and Applications (ICIEA), pp. 501–505 (2019). IEEE
22. Baena-Garcıa, M., Campo-Ávila, J., Fidalgo, R., Bifet, A., Gavalda, R., Morales-Bueno, R.: Early drift detection method. In: Fourth International Workshop on Knowledge Discovery from Data Streams, vol. 6, pp. 77–86. Citeseer (2006)
23. Museba, T., Nelwamondo, F., Ouahada, K., Akinola, A.: Recurrent adaptive classifier ensemble for handling recurring concept drifts. Appl. Comput. Intell. Soft Comput. **2021**, 1–13 (2021)
24. Gâlmeanu, H., Andonie, R.: Concept drift adaptation with incremental–decremental svm. Appl. Sci. (2021). https://doi.org/10.3390/app11209644
25. Bifet, A., Gavaldà, R.: Learning from time-changing data with adaptive windowing. In: Proceedings of the 2007 SIAM International Conference on Data Mining (SDM), pp. 443–448 (2007). https://doi.org/10.1137/1.9781611972771.42
26. Rubner, Y., Tomasi, C., Guibas, L.J.: The earth mover's distance as a metric for image retrieval. Int. J. Comput. Vision **40**(2), 99–121 (2000). https://doi.org/10.1023/A:1026543900054
27. Gama, J.A., Žliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. ACM Comput. Surv. (2014). https://doi.org/10.1145/2523813
28. Ali, U., Mahmood, T.: A novel framework for concept drift detection using autoencoders for classification problems in data streams. Int. J. Mach. Learn. Cybern. (2024). https://doi.org/10.1007/s13042-024-02223-2
29. Mehmood, T., Latif, S., Jamail, N.S.M., Malik, A., Latif, R.: Lstmdd: an optimized lstm-based drift detector for concept drift in dynamic cloud computing. PeerJ Comput. Sci. **10**, e1827 (2024)
30. Kraemer, F.A., Braten, A.E., Tamkittikhun, N., Palma, D.: Fog computing in healthcare-a review and discussion. IEEE Access **5**, 9206–9222 (2017)
31. Rahmani, A.M., Gia, T.N., Negash, B., Anzanpour, A., Azimi, I., Jiang, M., Liljeberg, P.: Exploiting smart e-health gateways at the edge of healthcare internet-of-things: A fog computing approach. Future Gener. Comput. Syst. **78**, 641–658 (2018)
32. Rincon, J.A., Guerra-Ojeda, S., Carrascosa, C., Julian, V.: An iot and fog computing-based monitoring system for cardiovascular patients with automatic ecg classification using deep neural networks. Sensors **20**(24), 7353 (2020)
33. Hameed Abdulkareem, K., Awad Mutlag, A., Musa Dinar, A., Frnda, J., Abed Mohammed, M., Hasan Zayr, F., Lakhan, A., Kadry, S., Ali Khattak, H., Nedoma, J.: Smart healthcare system for severity prediction and critical tasks management of covid-19 patients in iot-fog computing environments. Comput. Intell. Neurosci. **2022** (2022)
34. Sharma, M., Kaur, P.: Xlaam: explainable lstm-based activity and anomaly monitoring in a fog environment. J. Reliable Intell. Enviro., 1–15 (2022)
35. Tuli, S., Basumatary, N., Gill, S.S., Kahani, M., Arya, R.C., Wander, G.S., Buyya, R.: Healthfog: An ensemble deep learning based smart healthcare system for automatic diagnosis of heart diseases in integrated iot and fog computing environments. Futur. Gener. Comput. Syst. **104**, 187–200 (2020)
36. Alwateer, M., Almars, A.M., Areed, K.N., Elhosseini, M.A., Haikal, A.Y., Badawy, M.: Ambient healthcare approach with hybrid whale optimization algorithm and naive bayes classifier. Sensors **21**(13), 4579 (2021)
37. Chen, X., Wang, S., Zhang, T., Yang, G., Yang, Y.: Virtual concept drift detection and adaptation in federated data stream learning. Int. J. Data Sci. Anal. **21**(1), 46 (2026). https://doi.org/10.1007/s41060-025-00953-2
38. Diao, E., Ding, J., Tarokh, V.: Semifl: Communication efficient semi-supervised federated learning with unlabeled clients, vol. 3, (2021). arXiv:2106.01432 arXiv preprint
39. Casado, F.E., Lema, D., Iglesias, R., Regueiro, C.V., Barro, S.: Ensemble and continual federated learning for classification tasks. In: Machine Learning, pp. 1–41. (2023)
40. Jothimurugesan, E., Hsieh, K., Wang, J., Joshi, G., Gibbons, P.B.: Federated learning under distributed concept drift. In: International Conference on Artificial Intelligence and Statistics, pp. 5834–5853. (2023). (**PMLR**)
41. Kairouz, P., McMahan, H.B., Avent, B., Bellet, A., Bennis, M., Bhagoji, A.N., Bonawitz, K., Charles, Z., Cormode, G., Cummings, R., et al.: Advances and open problems in federated learning. Found. Trends Mach. Learn. **14**(1–2), 1–210 (2021)
42. Priya, S., Uthra, R.A.: Deep learning framework for handling concept drift and class imbalanced complex decision-making on streaming data. Complex Intell. Sys., 1–17 (2021)
43. Rajeswari, P.V.N., Shashi, M., Rao, T.K., Lakshmi, M.R., Kiran, L.V.: Effective intrusion detection system using concept drifting data stream and support vector machine. Concurrency and Computation: Practice and Experience **34**(21), 7118 (2022)
44. Kore, A., Bavil, E.A., Subasri, V., Abdalla, M., Fine, B., Dolatabadi, E., Abdalla, M.: Empirical data drift detection experiments on real-world medical imaging data. Nat. Commun. **15**(1), 1887 (2024). https://doi.org/10.1038/s41467-024-46142-w
45. Lukats, D., Zielinski, O., Hahn, A., Stahl, F.: A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams. Int. J. Data Sci. Anal. **19**, 1–31 (2025). https://doi.org/10.1007/s41060-024-00620-y
46. Assis, D.N., Souza, V.M.A.: Adwin-u: adaptive windowing for unsupervised drift detection on data streams. Knowl. Inf. Syst. **67**, 10005–10034 (2025). https://doi.org/10.1007/s10115-025-02523-1
47. He, Q., Xi, Z., Feng, Z., Teng, Y., Ma, L., Cai, Y., Zhang, H., Yu, K.: Telemedicine monitoring system based on fog/edge computing: A survey. IEEE Trans. Serv. Comput. **18**(1), 479–498 (2025). https://doi.org/10.1109/TSC.2024.3506473
48. Aguiar, G.J., Cano, A.: Class informed concept drift detection in multi-class data streams. Int. J. Data Sci. Anal. **21**(1), 42 (2026). https://doi.org/10.1007/s41060-025-00918-5
49. Ilic, A.S., Ilic, M., Stojanovic, N., Stojanovic, D.: Adaptive healthcare monitoring through drift-aware edge-cloud intelligence. Future Internet **18**(3), 156 (2026). https://doi.org/10.3390/fi18030156
50. Nizri, M.: COVID-19 Dataset. Kaggle dataset. Source: Government of Mexico, Secretaría de Salud, Dirección General de Epidemiología (DGE). Accessed: 2025-08-19 (2020). https://www.kaggle.com/datasets/meirnizri/covid19-dataset
51. Montiel, J., Read, J., Bifet, A., Abdessalem, T.: River: machine learning for streaming data in python. In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management, pp. 2451–2454 (2021)

Publisher"s Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.